

USENIX

THE ADVANCED COMPUTING SYSTEMS ASSOCIATION

HasteBoots: Proving TFHE Programmable Bootstrapping in Seconds

Fengrun Liu, *Carnegie Mellon University;* Haofei Liang, *Shanghai Jiao Tong University;* Xiang Xie, *East China Normal University and Primus Labs;* Yu Yu, *Shanghai Jiao Tong University;* Wenting Zheng, *Carnegie Mellon University;* Yuncong Hu, *Shanghai Jiao Tong University*

https://www.usenix.org/conference/usenixsecurity26/presentation/liu-fengrun

This paper is included in the Proceedings of the 35th USENIX Security Symposium.

August 12–14, 2026 • Baltimore, MD, USA

ISBN 978-1-939133-58-8

Open access to the Proceedings of the 35th USENIX Security Symposium is sponsored by



جامعة الملك عبدالله للعلوم والتقنية

King Abdullah University of Science and Technology

ARTIFACT EVALUATED
USENIX
AVAILABLE

HasteBoots: Proving TFHE Programmable Bootstrapping in Seconds

Fengrun Liu[1], Haofei Liang[2], Xiang Xie[3,4], Yu Yu[2], Wenting Zheng[1], Yuncong Hu[✉,2]

[1] *Carnegie Mellon University,* [2] *Shanghai Jiao Tong University*
[3] *East China Normal University,* [4] *Primus Labs*

Abstract

Fully Homomorphic Encryption (FHE) enables computations on encrypted data, ensuring privacy for outsourced computation. However, verifying the integrity of FHE computations remains a significant challenge, especially for bootstrapping, the most computationally intensive operation in FHE. Prior approaches, including zkVM-based solutions and general-purpose SNARKs, suffer from inefficiencies, with proof generation time ranging from several hours to days.

In this work, we propose HasteBoots, a succinct argument tailored for TFHE with programmable bootstrapping. By designing efficient protocols for arithmetic operations over quotient rings, HasteBoots achieves proof generation in a few seconds for TFHE evaluation with programmable bootstrapping, significantly outperforming the state-of-the-art, Zama (CCS'25), which requires half an hour. Moreover, HasteBoots supports batching multiple TFHE evaluations and bootstrappings, a feature that prior work cannot practically support due to prohibitive proving cost. HasteBoots can prove a batch of 16 operations within one minute, while maintaining succinct verification, requiring only 126 ms and a proof size of 0.28 MB. Our approach demonstrates the potential for scalable and efficient verifiable FHE, paving the way for practical, privacy-preserving computations.

1 Introduction

Fully Homomorphic Encryption (FHE) [8,9,16,18,22,23,29] is a groundbreaking cryptographic technology that enables computations to be performed directly on encrypted data, thereby preserving data confidentiality throughout the computation process. However, while FHE guarantees data confidentiality, it does not inherently provide any *integrity* guarantees on the computation. This limitation is particularly problematic in outsourced computations, where both the correctness of the computation and the privacy of the data must be ensured. Prior work [13, 14, 19, 36, 52, 57] shows that vulnerabilities in ciphertext manipulation may allow adversaries to modify encrypted data and even recover the client's secret key.

Succinct Non-Interactive Arguments of Knowledge (SNARKs) [4, 7, 15, 17, 25, 30, 31, 45, 53] provide a cryptographic solution by generating compact, verifiable proofs that attest to the correctness of a computation. By leveraging SNARKs to verify the correctness of FHE evaluations, one can ensure not only data confidentiality but also the integrity of outsourced computation.

However, FHE operations are notoriously complex, and directly proving their correctness using general-purpose SNARKs incurs prohibitive overhead. As a result, most prior work [6, 12, 24, 27] restricts attention to either extremely simple homomorphic encryption (HE) schemes, such as BV [10], or to leveled HE schemes [8, 9, 16, 23], which support only bounded-depth computation and fall short of supporting bootstrapping. In all current FHE schemes, bootstrapping is the key to reduce the accumulated noise in ciphertexts, thereby supporting arbitrary-depth *fully* homomorphic computation.

In this work, we focus on TFHE [5, 23], an FHE scheme with particularly lightweight and programmable bootstrapping (PBS). While general-purpose SNARKs can, in principle, be used to prove any computation–including TFHE PBS–they are not well-suited to the specific algebraic structures and execution patterns of FHE schemes. Indeed, as estimated in [34], a single TFHE/FHEW bootstrapping [5, 23] can translate into billions of R1CS constraints. A recent work [51] proposes using general-purpose SNARKs [25, 48] to prove a SNARK-friendly bootstrapping circuit derived from TFHE [5, 23], thereby reducing the proving time to about half an hour for a single bootstrapping operation. Nevertheless, such a proving cost remains impractical for real-world deployment.

The high overhead observed in prior work largely stems from the use of general-purpose SNARKs to prove FHE operations over quotient rings of the form $R_q = \mathbb{Z}_q[X]/(X^N+1)$. Emulating ciphertext arithmetic in R_q–in particular polynomial multiplications implemented via the Number Theoretic Transform (NTT)–already incurs substantial circuit complexity. This overhead is further exacerbated when complex ciphertext maintenance operations, such as modulus switching, blind rotation, and key switching, must be explicitly expressed

as arithmetic circuits, inevitably resulting in large circuits and substantial proving costs.

In this work, our goal is to construct a proof system specifically tailored to FHE with bootstrapping that can generate proofs in just **a few seconds**, making verifiable fully homomorphic encryption practical for scalable, privacy-preserving computation without relying on centralized trust.

1.1 Our Contributions

In this work, we present HasteBoots, a verifiable FHE system for TFHE programmable bootstrapping (PBS) that efficiently generates scalable and succinct non-interactive arguments for its correctness.

- **Scalable and Efficient PIOPs for Arithmetic over R_q.** We propose scalable and super-efficient polynomial interactive oracle proofs (PIOPs) for arithmetic operations over quotient rings $R_q = \mathbb{Z}_q[X]/(X^N+1)$, which naturally emulate FHE ciphertext arithmetic. Our constructions include a PIOP for verifying negacyclic NTT-equality relation, enabling verification of polynomial arithmetic in the NTT domain, as well as a PIOP for batched polynomial multiplications that directly captures arithmetic constraints over R_q. Both PIOPs achieve linear proving time and sublinear verification time by expressing target computations as relations between a Fourier-like matrix and vectors (or matrices), and proving such relations via efficient sum-check protocols that leverage the algebraic structure of the Fourier-like matrix. Notably, the PIOP for batched polynomial multiplications naturally extends to a general framework for proving arithmetic relations over R_q, with no need to commit to the NTT representations of ciphertext polynomials.
- **Novel PIOP Constructions for FHE Operations**. We further present novel PIOP constructions tailored to TFHE programmable bootstrapping (PBS), with a particular focus on blind rotation, the key noise-maintenance operation in PBS. Instead of proving blind rotation as a sequential circuit, we decompose it into stand-alone subroutines to extract repeated arithmetic operations over R_q, and then reconstruct the sequential structure by expressing the chaining of these subroutines as succinct arithmetic relations. This approach enables us to efficiently prove batched arithmetic constraints over R_q with linear-time provers, while achieving verification complexity that is sublinear in the depth of the underlying FHE circuit. For the remaining non-arithmetic operations involved in PBS–including modulus switching and digit decomposition–we reduce them to lookup relations over moderate-size tables, avoiding the need to emulate complex arithmetic circuits. These lookup relations can be efficiently proven using existing lookup arguments.
- **We implement and evaluate HasteBoots**. We implement HasteBoots in Rust, and our evaluation demonstrates that HasteBoots significantly outperforms the current state of the art, Zama [51], for proving TFHE evaluations with bootstrapping. Under the same FHE parameters and security level, Zama requires about half an hour to prove a single FHE evaluation with bootstrapping, whereas our approach, using the Dory PCS [35], completes the proof in around 10 seconds, with verifier time of 118 ms and proof size of 0.22 MB. Moreover, our system supports batched proofs for multiple FHE evaluations and bootstrappings, whereas prior work does not practically support such batched proofs. In particular, we can prove a batch size of 16 within one minute, with verifier time of 126 ms and proof size of 0.28 MB.

1.2 Overview of Techniques

Start Point: Linear-time Prover for Negacyclic NTT. TFHE computations frequently switch between the coefficient and evaluation representations of polynomials. Consequently, establishing the correctness of the underlying Fourier transformations is a fundamental requirement for proving TFHE arithmetic, which in practice is typically implemented via FFT over floating numbers. To align with the algebraic structure between FHE and SNARKs, most verifiable FHE works instead operate over the quotient rings $R_q = \mathbb{F}_q/(X^N+1)$, where q is chosen such that $2N \mid q-1$ to enable *negacyclic* Number Theoretic Transforms (NTTs) with $O(N \log N)$ complexity. The negacyclic NTT transforms a degree-N polynomial $f(X)$ into evaluations at the odd $2N$-th roots of unity, namely at the points $\{\omega^{2y+1}\}_{y\in[N]}$, where $\omega \in \mathbb{F}_q$ is a primitive $2N$-th root of unity. Previous work, such as [51], proves NTT by realizing them as $O(N \log N)$ circuits that implement the NTT, and then applying general-purpose SNARKs. As a result, the proving cost is inherently tied to the quasi-linear size of the underlying NTT circuits, leading to substantial prover overhead.

In contrast, we represent the negacyclic NTT equality as a matrix-vector multiplication relation, and design a specialized PIOP that proves this relation with a linear-time prover and a sublinear-time verifier. Concretely, the negacyclic NTT can be written as $\boldsymbol{a}^\star = F \cdot \boldsymbol{a}$, where $\boldsymbol{a}$ is the coefficient vector of $f(X)$, $\boldsymbol{a}^\star$ is the corresponding NTT evaluation vectors and F is a Fourier-like matrix with $F(\mathcal{Y},\mathcal{X}) = \omega^{(2\mathcal{Y}+1)\mathcal{X}}$ for $\mathcal{Y},\mathcal{X} \in [N]$. It suffices to ver ify the equality over their multilinear extensions: $\tilde{\boldsymbol{a}}^\star(y) = \sum_{x\in\{0,1\}^{\log N}} \tilde{\boldsymbol{a}}(x) \cdot \tilde{F}(y,x)$ for all $y \in \{0,1\}^{\log N}$, which can be reduced to the equality at a random point $\boldsymbol{u} \in \mathbb{F}_q$. However, the prover requires $O(N^2)$ time to evaluate the polynomial $\tilde{F}(\boldsymbol{u},x)$ for all $x \in \{0,1\}^{\log N}$ when running the sum-check protocol, which leads to an inefficient prover. One key insight is that this Fourier-like matrix F has only $2N$ distinct values arranged in a structured manner, so that the computation of $\tilde{F}(\boldsymbol{u},x)$ can be written as a grand product of $\log N$ terms, where i-th term takes over only 2^{i+1} distinct values for all x. This structure of computation enables a linear-time dynamic programming algorithm to evaluate all $\tilde{F}(\boldsymbol{u},x)$ in $O(N)$ time, leading to an overall linear-time prover.

Sublinear-Time Verifier for Arithmetic over R_q. With a linear-time prover for negacyclic NTT, we further construct protocols for general arithmetic relations over R_q that achieve linear-time proving and sublinear-time verification.

We first consider batched polynomial multiplications $\boldsymbol{c}_i(X) = \boldsymbol{a}_i(X) \cdot \boldsymbol{b}_i(X)$ with corresponding NTT evaluation vectors $\boldsymbol{a}_i^\star$, $\boldsymbol{b}_i^\star$, and $\boldsymbol{c}_i^\star$. A strawman approach to proving polynomial multiplications is to proceed in two steps. First, for each polynomial, a PIOP for negacyclic NTT equality is invoked to establish consistency between its coefficient $\boldsymbol{a}_i$ and its NTT-domain evaluation $\boldsymbol{a}_i^\star$. Second, given NTT-domain evaluations, prove the corresponding element-wise multiplication relation $\boldsymbol{c}_i^\star = \boldsymbol{a}_i^\star \circ \boldsymbol{b}_i^\star$ using the sum-check protocol. A major drawback of this strawman approach is that FHE evaluation typically involves a large number of polynomial multiplications. Although the sum-check protocols can be batched to reduce sum-check messages, the verifier still needs to query both the coefficient and evaluation oracles for each polynomial. As a result, the verifier cost scales linearly with the number of polynomials, and thus fails to achieve succinct verification.

To address this issue, we design a batched polynomial multiplication protocol. Concretely, we organize the coefficients of the polynomials into matrices $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$, where each row corresponds to one polynomial, and let $\mathbf{A}^\star$, $\mathbf{B}^\star$, and $\mathbf{C}^\star$ denote their corresponding evaluation matrices of the NTT-domain. We observe that the negacyclic NTT equality relation can be extended to these matrices satisfying $\mathbf{A}^\star = F \cdot \mathbf{A}$, $\mathbf{B}^\star = F \cdot \mathbf{B}$, and $\mathbf{C}^\star = F \cdot \mathbf{C}$, where F is the Fourier-like matrix. Therefore, polynomial multiplications can be verified directly in the NTT domain via the Hadamard relation $\mathbf{A}^\star \circ \mathbf{B}^\star = \mathbf{C}^\star$. This batched formulation allows one to design succinct PIOPs for both the batched NTT equalities and the Hadamard relation, significantly reducing verifier overhead.

Moreover, we observe that NTT equality checks can be extended to evaluation proofs for the NTT-domain matrices, with the random challenges instantiated as evaluation points. In particular, after a sum-check protocol is used to prove the Hadamard relation and reduce the claim to evaluations of the matrices $\mathbf{A}^\star, \mathbf{B}^\star, \mathbf{C}^\star$, the resulting evaluation point can be used to instantiate the random challenges in the NTT equality PIOP. As a result, the verifier ultimately only needs to query polynomial coefficient oracles $\mathbf{A}, \mathbf{B}, \mathbf{C}$.

Notably, this construction for batched polynomial multiplications naturally extends to a general framework for proving batched arithmetic relations over R_q. It offers three key advantages: (1) it proves arithmetic relations directly over R_q with a linear-time prover, rather than explicitly proving arithmetic operations via $O(N \log N)$ NTT circuits; (2) it achieves a succinct verifier when proving batched arithmetic relations, thereby preserving succinctness; and (3) it does not require committing to NTT representations of polynomials, which further improves prover efficiency compared to prior work.

Proving Arithmetic Relations in TFHE PBS. A TFHE PBS procedure typically consists of four main stages: modulus switching, blind rotation, key switching, and sample extraction. We focus here on the arithmetic relations arising in blind rotation, as it forms the core of TFHE PBS and constitutes the main bottleneck in proving complexity. The remaining arithmetic operations are fully addressed in Section 4.

Blind rotation is implemented as a long sequence of repeated sub-circuits, each consisting of arithmetic operations over R_q interleaved with non-arithmetic steps, such as digit decomposition. Hence, we need to prove both the correctness of these sub-circuits and their sequential chaining structure. To enable an efficient prover while maintaining succinct verification, we extract these repeated arithmetic operations over R_q and apply our previously introduced PIOP framework to prove the resulting batched arithmetic relations, yielding verifier time sublinear in the circuit depth. Moreover, we observe that the chaining of these sub-circuits can be captured by a linear relation defined by a prescribed matrix, which can be efficiently verified using sum-check protocols.

Concretely, blind rotation involves two types of arithmetic relations. The first consists of sums of polynomial products, i.e., $\boldsymbol{c}(X) = \sum_{i=0}^{2\ell} \boldsymbol{a}_i(X) \cdot \boldsymbol{b}_i(X)$, which can be handled efficiently by our efficient PIOPs for arithmetic over R_q. The second is the negacyclic rotation of a polynomial by an offset a, which is algebraically equivalent to multiplying the monomial X^a in R_q. Our key observation is that the coefficient-domain matrix of the batched monomials is sparse, and its sparse representation is directly determined by rotation offsets. This allows us to reduce negacyclic rotations to polynomial multiplications by monomials within our PIOP framework, with verifier queries further reduced to evaluations on their rotation offsets using techniques from sparse polynomial commitment schemes [45, 46].

To prove the chaining among sub-circuits, we batch the input and output polynomials into matrices C_{in} and C_{out}, with one row per sub-circuit, the chaining constraints can be written in the form $\mathsf{C}_{\text{out}} = P \cdot \mathsf{C}_{\text{in}} + \mathsf{Z}$, where P is a prescribed row-permutation matrix enforcing the shift between consecutive sub-circuits, and Z is a difference matrix that captures the difference between the beginning and end of the chain. This linear structure allows the chaining correctness to be verified efficiently alongside the arithmetic relations.

Proving Non-Arithmetic Relations in TFHE PBS. In addition to arithmetic operations, TFHE PBS involves two types of non-arithmetic operations: digit decomposition in blind rotation and key switching, and modulus switching. Our design principle is to avoid expressing these operations as arithmetic circuits, which would otherwise lead to large circuit sizes and high proving costs. Instead, we design lookup relations for non-arithmetic operations, while keeping the lookup table size moderate. These lookup relations can then be efficiently proven using existing lookup techniques. The detailed constructions are presented in Section 5.

1.3 Related Work

SNARK for FHE. Many prior works attempt to prove the correctness of FHE computations using SNARKs, but proving ciphertext arithmetic over $R_q = \mathbb{Z}_q/(X^N+1)$ remains inefficient. In particular, some approaches [6, 12] design sum-check–based proof systems that operate directly over quotient rings to natively capture arithmetic over R_q, but incur significant NTT computation overhead in sum-check protocols and polynomial commitment schemes. Other works [24, 27, 51] translate ciphertext arithmetic in R_q into explicit arithmetic circuits or R1CS constraints over $\mathbb{Z}_q$ to use general-purpose SNARKs, which inevitably incur costs of explicitly proving the NTT-based circuits underlying polynomial multiplication. Despite their different formulations, both approaches introduce quasi-linear proof-generation overhead stemming from the inherent cost of NTT-based polynomial computations in R_q. In contrast, we propose efficient PIOPs that directly prove NTT-equality relations of polynomials in R_q, which can be further utilized to achieve linear-time provers and sublinear-time verifiers for arithmetic relations over R_q, without explicitly proving NTT circuits or committing to NTT representations.

In addition, most prior works [6, 24, 27] only support simple constant-depth homomorphic encryption schemes–BV [10] and BGV/BFV [8, 9, 23]–and fall short in handling non-arithmetic ciphertext maintenance operations due to the inefficiency of expressing such operations using arithmetic circuits or R1CS constraints. Recent theoretical work [12] employs lookup-based techniques to support limited noise-management operations in leveled CKKS [16], but still falls short of supporting bootstrapping, which is essential for fully homomorphic evaluation. The only recent work [51] that supports TFHE bootstrapping [5, 23] continues to express all FHE operations as arithmetic circuits, including non-arithmetic ones, resulting in substantial proving overhead. In contrast, our work presents new constructions for TFHE programmable bootstrapping [5, 23] that separate the treatment of structured arithmetic relations from non-arithmetic relations. This design enables efficient proof generation by proving arithmetic operations using our PIOPs over R_q, while reducing non-arithmetic operations to lookup-based relations handled by existing efficient lookup arguments.

FHE for SNARK. Another line of work [3, 26, 28, 42, 56] follows a fundamentally different paradigm from directly proving FHE operations with SNARKs. Instead, these works homomorphically generate SNARKs for the plaintext computation, aiming to achieve a comparable integrity guarantee. However, prior works [3, 56] show that this approach inevitably leaks at least one bit of information about the plaintext and thereby undermines the IND-CPA security guarantees of FHE. Moreover, since the proof itself is an encrypted ciphertext, verification can only be performed by the holder of the decryption key, thereby preventing public verifiability.

2 Preliminary

2.1 Notation

For an integer N, $[N]$ denotes the set $\{0,1,\ldots,N-1\}$. Let $\mathbb{F}$ be a finite field of prime order Q. Vectors and matrices on $\mathbb{F}$ are denoted by bold lowercase and bold uppercase letters, i.e. $\boldsymbol{a}$ and $\mathbf{A}$, respectively. We write $\boldsymbol{a} \leftarrow \mathbb{F}^N$ to denote a vector sampled uniformly from $\mathbb{F}^N$. The Hadamard product of vectors or matrices is defined as element-wise multiplication and is denoted by $\boldsymbol{a} \circ \boldsymbol{b}$ and $\mathbf{A} \circ \mathbf{B}$, respectively. Let $\mathbf{1}\{\cdot\}$ denote the indicator function.

Bit-Reversed Representation. Assume N is a power of two. For $x = (x_0,\ldots,x_{\log N-1}) \in \{0,1\}^{\log N}$, we identify its canonical field encoding as $\mathcal{X} = \sum_{i=0}^{\log N-1} 2^i x_i \in \mathbb{F}$. We write $\text{to-bits}(\mathcal{X})$ for the inverse map from $\mathcal{X} \in \mathbb{F}$ to its binary representation $x \in \{0,1\}^{\log N}$. When $\mathcal{X}$ denotes an index, its bit-reversal is defined as $\mathcal{X}^{\mathsf{R}} = \sum_{i=0}^{\log N-1} 2^{\log N-1-i} x_i$. Accordingly, for a vector $\boldsymbol{a} = (a_0,\ldots,a_{N-1})$, we define its bit-reversed order as $\boldsymbol{a}^{\mathsf{R}} = (a_{0^{\mathsf{R}}},\ldots,a_{(N-1)^{\mathsf{R}}})$.

Multilinear Extensions of Vectors and Matrices. Let the identity function $eq : \{0,1\}^{\log N} \times \{0,1\}^{\log N} \to \{0,1\}$ be defined as $eq(x,y) = 1$ if $x = y$, and 0 otherwise. Its multilinear extension, denoted $\tilde{eq}(x,y)$, is given by $\tilde{eq}(x,y) = \prod_{i=1}^{\log N}((1-x_i)(1-y_i) + x_i y_i)$ for $x,y \in \mathbb{F}^{\log N}$. A vector $\boldsymbol{a} \in \mathbb{F}^N$ can be viewed as a function $\boldsymbol{a} : \{0,1\}^{\log N} \to \mathbb{F}$ such that $\boldsymbol{a}(x) = a_{\mathcal{X}}$ for all $x \in \{0,1\}^{\log N}$. Its multilinear extension $\boldsymbol{a} : \mathbb{F}^{\log N} \to \mathbb{F}$ is uniquely defined as $\tilde{\boldsymbol{a}}(x) = \sum_{b \in \{0,1\}^{\log N}} \tilde{eq}(x,b) \cdot \boldsymbol{a}(b)$, which satisfies $\tilde{\boldsymbol{a}}(x) = \boldsymbol{a}(x)$ for all $x \in \{0,1\}^{\log N}$. Similarly, a matrix $\mathbf{A} \in \mathbb{F}^{M \times N}$ can be viewed as a function $\mathbf{A} : \{0,1\}^{\log M} \times \{0,1\}^{\log N} \mapsto \mathbb{F}$ such that $\mathbf{A}(x,y) = \mathbf{A}_{\mathcal{X},\mathcal{Y}}$. Its multilinear extension $\tilde{\mathbf{A}}$, is defined as $\tilde{\mathbf{A}}(x,y) = \sum_{(b,c) \in \{0,1\}^{\log M + \log N}} \tilde{eq}(x,b) \cdot \tilde{eq}(y,c) \cdot \mathbf{A}(b,c)$ and satisfies $\tilde{\mathbf{A}}(x,y) = \mathbf{A}(x,y)$ for all $(x,y) \in \{0,1\}^{\log M + \log N}$.

Cyclotomic Ring. For N a power of two, the $2N$-th cyclotomic ring is defined as $\mathcal{R} = \mathbb{Z}[X]/(X^N+1)$. Its quotient ring $\mathcal{R}_Q = \mathcal{R}/(Q\mathcal{R})$ is the standard polynomial ring used in RLWE-based FHE schemes. Since throughout this paper $\mathbb{F}$ denotes a finite field of prime order Q, the ring $\mathcal{R}_Q$ is isomorphic to $\mathbb{F}[X]/(X^N+1)$. Accordingly, any polynomial $\boldsymbol{c}(X) = \sum_{i=0}^{N-1} c_i X^i \in R_Q$ can be uniquely represented by its coefficient vector $\boldsymbol{c} = (c_0,\ldots,c_{N-1}) \in \mathbb{F}^N$. When there is no ambiguity, we abuse notation and use $\boldsymbol{c}$ to denote the polynomial $\boldsymbol{c}(X)$ itself.

Negacyclic NTT. The Number Theoretic Transform (NTT) is the finite-field analogue of FFT, enabling fast polynomial multiplication via element-wise products. Since we work over the cyclotomic ring $R_Q = \mathbb{F}[X]/(X^N+1)$, we use the *negacyclic* NTT. Throughout this paper, we assume that $2N \mid (Q-1)$, so that $\mathbb{F}$ contains a primitive $2N$-th root of unity, denoted by ω. The NTT maps a polynomial $\boldsymbol{c}(X) \in R_Q$, represented by its coefficient vector $\boldsymbol{c}$, to a unique NTT vector $\boldsymbol{c}^\star$, consisting of evaluations of $\boldsymbol{c}(X)$ at the N distinct roots of X^N+1,

namely, $\omega^{2\mathcal{Y}+1}$ for $\mathcal{Y} \in [N]$. Then, for polynomial multiplication $\boldsymbol{c}(X) = \boldsymbol{a}(X) \cdot \boldsymbol{b}(X)$, it holds that $\boldsymbol{c}^\star = \boldsymbol{a}^\star \circ \boldsymbol{b}^\star$.

Extension Field. Let $F(X) \in \mathbb{F}[X]$ be an irreducible polynomial of degree D. We define the extension field $\mathbb{EF} = \mathbb{F}[X]/(F)$, which has size of Q^D. Elements of $\mathbb{EF}$ are represented as polynomials of degree at most $D-1$, and arithmetic in $\mathbb{EF}$ is performed via polynomial arithmetic modulo $F(X)$.

2.2 Fully Homomorphic Encryption

In this work, we focus on the TFHE [5, 18, 20] with fast programmable bootstrapping procedures.

Ciphertext Types. There are two main types of ciphertexts: Learning with Errors (LWE) [43] and Ring Learning with Errors (RLWE) [40] ciphertexts. An LWE ciphertext encrypting a message $m \in \mathbb{Z}_t$ has the form: $(\boldsymbol{a}, b) = (\boldsymbol{a}, \langle \boldsymbol{a}, \boldsymbol{s} \rangle + \Delta \cdot m + e \bmod Q)$ where $\boldsymbol{a} \leftarrow \mathbb{F}^n$ is sampled uniformly, $\boldsymbol{s} \in \{0,1\}^n$ is the secret key, e is the noise term $\Delta = \lfloor Q/t \rceil$ is a scaling factor.

A RLWE ciphertext encrypting a polynomial messgage $\boldsymbol{m}(X)$ has the form: $(\boldsymbol{a}(X), \boldsymbol{b}(X)) = (\boldsymbol{a}(X), \boldsymbol{a}(X) \cdot \boldsymbol{z}(X) + \Delta \boldsymbol{m}(x) + \boldsymbol{e}(X))$ where $\boldsymbol{a}(X) \leftarrow \mathcal{R}_Q$, $\boldsymbol{z}(X)$ is the secret polynomial, and $\boldsymbol{e}(X)$ is sampled from a discrete gaussian distribution. RLWE ciphertext addition is defined element-wise and is homomorphic with respect to message addition. An RLWE ciphertext can be represented in its coefficient vectors, so we use $\mathbb{RLWE}$ to denote the space $\mathbb{F}^N \times \mathbb{F}^N$.

In RLWE-based FHE schemes, bootstrapping commonly relies on two derived ciphertext types: RLev ciphertexts and RGSW ciphertexts. Fixed a decomposition base B and let $\ell = \lceil \log_B Q \rceil$. An RLev ciphertext encrypting a message $\boldsymbol{m}$ is a tuple $\mathsf{k} = \{\mathsf{k}_i\}_{i=0}^{\ell-1} \in \mathbb{RLWE}^\ell$, where each $\mathsf{k}_i = (\mathsf{k}_i.\boldsymbol{a}, \mathsf{k}_i.\boldsymbol{b})$ is an RLWE ciphertext encrypting $B^i \boldsymbol{m}$ under the same secret key. An RGSW ciphertext encrypting $\boldsymbol{m}$ consists of two RLev ciphertexts $(\mathsf{k}, \mathsf{k}')$, encrypting $-\boldsymbol{z}\boldsymbol{m}$ and $\boldsymbol{m}$, respectively. We denote the space of such ciphertexts by $\mathbb{RGSW} = \mathbb{RLWE}^\ell \times \mathbb{RLWE}^\ell$, each represented by 4ℓ vectors in $\mathbb{F}^N$.

Ciphertext Multiplications. We briefly recall the ciphertext multiplication operations used in TFHE, and emphasize that all of them reduce to (a sum of) polynomial products over R_Q.

Let $\boldsymbol{e}$ be a B-bounded polynomial, and let $\mathsf{c} = (\boldsymbol{a}, \boldsymbol{b})$ be an RLWE ciphertext that encrypts $\boldsymbol{m}$. We define $\boldsymbol{e} \odot \mathsf{c} = (\boldsymbol{e} \cdot \boldsymbol{a}, \boldsymbol{e} \cdot \boldsymbol{b})$. The resulting ciphertext encrypts the message $\boldsymbol{e}\boldsymbol{m}$.

To support multiplication by an arbitrary polynomial, we define the operation $(\otimes)$ with RLev ciphertexts. Let $\boldsymbol{d} = \sum_{i=0}^{\ell-1} B^i \boldsymbol{d}_i$, where each $\boldsymbol{d}_i$ is B-bounded, and let $\mathsf{k} = \{\mathsf{k}_i\}_{i=0}^{\ell-1}$ be an RLev ciphertext. We define $\boldsymbol{d} \otimes \mathsf{k} = \sum_{i=0}^{\ell-1} \boldsymbol{d}_i \odot \mathsf{k}_i$, which results in an RLWE ciphertext encrypting $\boldsymbol{d}\boldsymbol{m}$.

Finally, we define the *external product* $(\diamond)$ between an RLWE ciphertext and RGSW ciphertext. Given an RLWE ciphertext $\mathsf{c} = (\boldsymbol{a}, \boldsymbol{b})$ and an RGSW ciphertext $\mathsf{K} = (\mathsf{k}, \mathsf{k}')$, the external product is defined as $\mathsf{c} \diamond \mathsf{K} = \boldsymbol{a} \otimes \mathsf{k} + \boldsymbol{b} \otimes \mathsf{k}'$, which results in an RLWE ciphertext encrypting the product $\boldsymbol{m}_0 \cdot \boldsymbol{m}_1$.

In summary, $(\otimes)$ consists of 2ℓ polynomial multiplications and $(\diamond)$ consists of 4ℓ polynomial multiplications in R_Q.

Algorithm 1 PBS

Params: $B \in \mathbb{Z}$ and $\ell = \lceil \log_B Q \rceil$
Eval Keys: $\mathsf{bsk} = \{\mathsf{K}_i\}_{i=0}^n \in \mathbb{RGSW}^n$ and $\mathsf{ksk} \in \mathbb{RLWE}^\ell$
Lookup Table: $\mathsf{lut} \in R_Q$ related to a function f

Input: $(\boldsymbol{a}, b) \in \mathbb{F}^{n+1}$ encrypting a message $m \in \mathbb{Z}_t$
Output: $\mathsf{ct} \in \mathbb{F}^{n+1}$ encrypting a message $f(m) \in \mathbb{Z}_t$

// Modulus Switching
$a'_i \leftarrow \lfloor \frac{a_i \cdot 2N}{Q} \rceil \in \mathbb{Z}_{2N}$ for $i = 0, \ldots, n-1$
$b' \leftarrow \lfloor \frac{b \cdot 2N}{Q} \rceil \in \mathbb{Z}_{2N}$
// Blind Rotation
$\mathsf{acc}[0] \leftarrow (\boldsymbol{0}, \mathsf{lut} \cdot X^{-b'}) \in \mathbb{RLWE}$
for $i = 0$ to $n-1$ **do**
  $\mathsf{h}_i \leftarrow X^{a'_i} \odot \mathsf{acc}[i] - \mathsf{acc}[i] \in \mathbb{RLWE}$
  $\mathsf{d}_i \leftarrow \mathsf{h}_i \diamond \mathsf{K}_i \in \mathbb{RLWE}$ // External Product
  $\mathsf{acc}[i+1] \leftarrow \mathsf{acc}[i] + \mathsf{d}_i \in \mathbb{RLWE}$
end for
// Key Switching
$(\boldsymbol{a}'', \boldsymbol{b}'') \leftarrow \mathsf{acc}[n]$
$\mathsf{init} \leftarrow (\boldsymbol{0}, \boldsymbol{b}'')$
$\delta \leftarrow \boldsymbol{a}'' \otimes \mathsf{ksk} \in \mathbb{RLWE}$
$(\boldsymbol{\alpha}, \boldsymbol{\beta}) \leftarrow \mathsf{init} - \delta \in \mathbb{RLWE}$
// Sample Extraction
$\boldsymbol{\alpha}' \leftarrow (\alpha_0, -\alpha_{N-1}, \ldots, -\alpha_1) \in \mathbb{F}^N$
return $\big((\alpha'_0, \ldots, \alpha'_{n-1}), \beta_0\big) \in \mathbb{F}^{n+1}$

Overview of TFHE PBS. TFHE programmable bootstrapping (PBS) takes as input:

- an LWE ciphertext $(\boldsymbol{a}, b = \langle \boldsymbol{a}, \boldsymbol{s} \rangle + e + \Delta \cdot m)$;
- a polynomial $\mathsf{lut} \in R_Q$, encoding a function $f : \mathbb{Z}_t \to \mathbb{Z}_t$;
- the bootstrapping key $\mathsf{bsk} = \{\mathsf{K}_i\}_{i=0}^{n-1}$, where each K_i is a RGSW ciphertext encrypting the secret key bit s_i;
- a key switching key ksk represented as an RLev ciphertext.

The output is a refreshed LWE ciphertext encrypting $f(m)$ under the original secret key $\boldsymbol{s}$, with noise independent of the input ciphertext.

We follow the PBS procedure from [51], which consists of four steps: modulus switching, blind rotation, key switching, and sample extraction (see Algorithm 1). We refer readers to the full version [37] and [18, 41] for full details.

2.3 Polynomial IOPs

Indexed Oracle Relation. An indexed relation $\mathcal{R}$ is a set of tuples $(\mathbf{i}, \mathbf{x}; \mathbf{w})$ where $\mathbf{i}$ is the index (omitted when empty), $\mathbf{x}$ the statement and $\mathbf{w}$ the witness. Following the notion of indexed oracle relations from [15], an indexed oracle relation $\mathcal{R}$ is an indexed relation in which the index $\mathbf{i}$ and the statement $\mathbf{x}$ include pointers to oracle polynomials, with the polynomials themselves being part of the witness $\mathbf{w}$. A pointer to a polynomial p is denoted by $[\![p]\!]$. The formal definition of polynomial interactive oracle proofs (PIOP) with indexed oracle relations is given in [12] and recalled in the full version [37].

Virtual Oracles. We adopt the notion of virtual oracles from [15]. For an arithmetic function $\mathcal{G}$, a virtual oracle $[\![p]\!]$ for $\mathcal{G}([\![p_1]\!],\ldots,[\![p_k]\!])$ consists of the list of oracles $[\![p_1]\!],\ldots,[\![p_k]\!]$, together with a description of how to compute $\mathcal{G}$ from these oracles. A query of a virtual oracle $[\![p]\!]$ at any point r is answered by first querying each oracle $[\![p_i]\!]$ at point r to obtain $p_i(r)$, and then computing $\mathcal{G}(p_1(r),\ldots,p_k(r))$.

Sum-check Protocol. Let $p(x):\mathbb{F}^\mu\to\mathbb{F}$ be a μ-variate polynomial of low degree. The sum-check relation $\mathcal{R}_{\mathsf{Sum}}$ consists of tuples $((T,[\![p]\!]);p)$ such that $T=\sum_{x\in\{0,1\}^\mu}p(x)$. The sum-check protocol [39] enables an untrusted prover $\mathcal{P}$ to convince a verifier $\mathcal{V}$ of the correctness of this claim. After μ rounds of interactions, the verification reduces to a single query to the virtual oracle $[\![p]\!]$ at a random point $r\leftarrow\mathbb{F}^\mu$. This protocol has $O(2^\mu)$ prover time and logarithmic verifier time and round complexity using the standard algorithm [49, 53] to compute the messages of all rounds. A formal description of the sum-check protocol Π_{Sum} is provided in Section B.1.

Zero Check. The sum-check protocol can be used to prove the zero-check relation $\mathcal{R}_{\mathsf{Zero}}$, which consists of tuples $([\![p]\!];p)$ such that $p(x)=0$ for all $x\in\{0,1\}^\mu$, where p is typically given as a virtual oracle as defined above. The formal relation and PIOP protocol Π_{Zero} for $\mathcal{R}_{\mathsf{Zero}}$ are provided in the full version [37].

Lookup. Lookup arguments are used to prove that a sequence of values belongs to a table, which is often prescribed. We define the lookup relation $\mathcal{R}_{\mathsf{Lookup}}$ as the set of tuples $((M,N,[\![t]\!]),([\![f]\!]);f,t)$ such that $\{f(x):x\in\{0,1\}^{\log N}\}\subseteq\{t(x):x\in\{0,1\}^{\log M}\}$ as sets. In this work, we instantiate lookup arguments using the construction based on logarithmic derivatives due to Haböck [32]. This notion naturally extends to indexed lookups relation $\mathcal{R}^*_{\mathsf{Lookup}}$ consists of $((M,N,[\![t]\!]),([\![f]\!],[\![\mathsf{id}]\!]);f,t,\mathsf{id})$ such that $f(x)=t(\mathsf{to\text{-}bits}(\mathsf{id}(x)))$ for all $x\in\{0,1\}^{\log N}$. More details about the formal definitions and PIOPs are provided in the full version [37].

2.4 PCS and Compilation to SNARKs

Polynomial commitment schemes (PCS) allow a prover to commit to a polynomial and later prove evaluations of the form $y=p(r)$. Using standard compilation theorems [11, 17], a PIOP for an indexed oracle relation can be compiled into a public-coin interactive argument of knowledge by replacing oracle queries with polynomial commitments and opening proofs. Applying the Fiat–Shamir transform further yields a succinct non-interactive argument of knowledge (SNARK). Formal definitions are provided in the full version [37].

3 PIOP for Proving Arithmetic in R_Q

In this section, we develop an efficient PIOP framework for proving ciphertext arithmetic relations arising in TFHE bootstrapping. The key to this framework is a set of prover-efficient PIOPs for verifying negacyclic NTT equality over single and batched polynomials, described in Section 3.1 and Section 3.2, respectively. Building on these, we present our main PIOP for batched polynomial multiplication in Section 3.3, which outlines a general framework for proving arithmetic relations in R_Q. Finally, we present a specialized protocol in Section 3.4 for handling the negacyclic rotations in TFHE.

3.1 Negacyclic NTT Equality

NTT is the key primitive enabling efficient polynomial multiplication over the ring R_Q in practice. In this section, we present a specialized PIOP for proving the correctness of the negacyclic NTT in the cyclotomic ring $R_Q=\mathbb{F}[X]/(X^N+1)$. At a high level, our protocol extends the cyclic FFT verification approach of zkCNN [38], which operates over $\mathbb{F}[X]/(X^N-1)$, to the negacyclic NTT setting required by FHE polynomial rings.

This extension is nontrivial because the underlying evaluation domain changes. The cyclic FFT considered in zkCNN [38] evaluates over the roots of X^N-1, namely $\omega,\omega^2,\ldots,\omega^N$, whereas the negacyclic NTT evaluates over the roots of X^N+1, namely the odd powers $\omega,\omega^3,\ldots,\omega^{2N-1}$. This change alters the structure of the Fourier-like matrix and prevents us from directly applying zkCNN's bookkeeping-table algorithm and protocol to the negacyclic NTT setting. Instead, we introduce a new bookkeeping-table algorithm and a new delegation protocol specialized to the negacyclic NTT matrix, achieving linear prover time and sublinear verification costs without committing to the structured Fourier-like matrix.

In addition to the negacyclic evaluation domain, we consider a practical NTT implementation in which the output NTT vector is arranged in bit-reversed order. This design choice allows the prover to directly reuse the computation trace produced by existing implementations, without introducing an additional reordering step.

Under this convention, the negacyclic NTT maps a polynomial's coefficient vector $\boldsymbol{c}\in\mathbb{F}^N$ to an evaluation vector $\boldsymbol{a}\in\mathbb{F}^N$ by evaluating the polynomial at the points $X=\omega^{2\mathcal{Y}+1}$ for $\mathcal{Y}\in[N]$. We denote the resulting bit-reversed evaluation vector by $\boldsymbol{a}^{\mathsf{R}}$. Concretely, the $\mathcal{Y}$-th entry of $\boldsymbol{a}^{\mathsf{R}}$ corresponds to the evaluation at $X=\omega^{2\mathcal{Y}^{\mathsf{R}}+1}$. With this notation, the negacyclic NTT can be expressed as a matrix–vector multiplication $\boldsymbol{a}^{\mathsf{R}}=F^{\mathsf{R}}\cdot\boldsymbol{c}$, where F^{R} is a Fourier-like matrix defined by $F^{\mathsf{R}}(\mathcal{Y},\mathcal{X})=\omega^{(2\mathcal{Y}^{\mathsf{R}}+1)\cdot\mathcal{X}}$, which explicitly incorporates the bit-reversal permutation.

To prove the correctness of this matrix–vector multiplication, we rewrite it as the following multilinear polynomial identity:

$$\tilde{\boldsymbol{a}}^{\mathsf{R}}(y)=\sum_{x\in\{0,1\}^{\log N}}\tilde{\boldsymbol{c}}(x)\cdot\tilde{F}^{\mathsf{R}}(y,x),\tag{1}$$

Algorithm 2 $\mathsf{F} \leftarrow \mathsf{Fourier}(\boldsymbol{u})$
Params: N and the primitive $2N$-th root of unity ω in R_Q.

Input: The point $\boldsymbol{u} \in \mathbb{EF}^{\log N}$.
Output: F where $\mathsf{F}[\mathcal{X}] = \tilde{F}^{\mathsf{R}}(\boldsymbol{u}, x)$ for all $x \in \{0,1\}^{\log N}$

```
F[0] = 1
for i = 0, ..., log N − 1 do
    for j = 2^{i+1} − 1, ..., 0 do
        b = (j >> i)&1     // the i-th bit of j
        F[j] = F[j%2^i] · ((1 − u_i) + u_i · ω^j_{2^{i+1}}) · ω^{2^i · b}
return F
```

which holds for all $y \in \{0,1\}^{\log N}$.

By the Schwartz–Zippel lemma, it suffices to check this identity at a random point $\boldsymbol{u} \in \mathbb{EF}^{\log N}$, yielding soundness error at most $\log N/|\mathbb{F}|$. The prover computes $T = \tilde{\boldsymbol{a}}^{\mathsf{R}}(\boldsymbol{u})$ and obtains a virtual oracle $[\![\mathcal{G}]\!]$ with underlying polynomial $\mathcal{G}(x) = \tilde{\boldsymbol{c}}(x) \cdot \tilde{F}^{\mathsf{R}}(\boldsymbol{u}, x)$. Here $[\![\mathcal{G}]\!]$ is derived algebraically from the existing oracles $[\![\boldsymbol{c}]\!]$ and $[\![F^{\mathsf{R}}]\!]$. The prover and verifier then execute a sum-check protocol for the relation $((T, [\![\mathcal{G}]\!]); \boldsymbol{c}, \tilde{F}^{\mathsf{R}}(\boldsymbol{u}, \cdot)) \in \mathcal{R}_{\mathsf{Sum}}$.

However, from the prover's perspective, a direct construction of the virtual oracle $[\![\mathcal{G}]\!]$ is computationally expensive. Indeed, defining $\mathcal{G}(x) = \tilde{\boldsymbol{c}}(x) \cdot \tilde{F}^{\mathsf{R}}(\boldsymbol{u}, x)$ requires evaluating the polynomial $\tilde{F}^{\mathsf{R}}(\boldsymbol{u}, x)$ at all $x \in \{0,1\}^{\log N}$. As implied by Lemma 4.4 of [50], a naive computation of these values takes $O(N^2)$ time. Fortunately, the Fourier-like matrix F^{R} underlying the negacyclic NTT contains only $2N$ distinct values and exhibits a highly structured form closely related to the standard Fourier matrix, which contains N distinct values. Inspired by the standard Fourier matrix evaluation technique from [38], we develop an evaluation algorithm specialized for the NTT matrix. By exploiting its inherent algebraic structure, our protocol computes all values of $\tilde{F}^{\mathsf{R}}(\boldsymbol{u}, x)$ in $O(N)$ time.

For each $x \in \{0,1\}^{\log N}$, we derive a grand-product representation of $\tilde{F}^{\mathsf{R}}(\boldsymbol{u}, x)$ of the following form. The full derivation is provided in Section A.2.

$$\tilde{F}^{\mathsf{R}}(\boldsymbol{u}, x) = \prod_{i=0}^{\log N - 1} \left(1 - u_i + u_i \cdot \omega_{2^{i+1}}^{\mathcal{X}}\right) \cdot \omega^{2^i \cdot x_i}. \tag{2}$$

Here, $\omega_{2^{i+1}}$ denotes a primitive 2^{i+1}-th root of unity, so that $\omega_{2^{i+1}}^{\mathcal{X}}$ ranges over 2^{i+1} distinct values as $\mathcal{X}$ varies over $[N]$.

Crucially, this structure allows the computation of $\tilde{F}^{\mathsf{R}}(\boldsymbol{u}, x)$ to be divided into $\log N$ rounds. At round i, the factor involving $\omega_{2^{i+1}}^{\mathcal{X}}$ only depends on 2^{i+1} distinct values, enabling a dynamic programming approach. Finally, the values of $\tilde{F}^{\mathsf{R}}(\boldsymbol{u}, x)$ for all $x \in \{0,1\}^{\log N}$ can be computed in overall $O(N)$ time, as illustrated in Algorithm 2. At the end of the sumcheck, the verifier queries $[\![F^{\mathsf{R}}]\!]$ and $[\![\boldsymbol{c}]\!]$ at a random point.

We now formalize the NTT-equality relation $\mathcal{R}_{\mathsf{NTT}}$ in Definition 1 and present a corresponding PIOP, denoted by Π_{NTT}, which achieves $O(N)$ prover time and $O(\log N)$ verifier time. The full protocol description is provided in Section B.2.

Def 1. *The NTT-Equality relation $\mathcal{R}_{\mathsf{NTT}}$ is the set of tuples*

$$(\mathbf{i}, \mathbf{x}, \mathbf{w}) = ((N, [\![F^{\mathsf{R}}]\!]), ([\![\boldsymbol{c}]\!], [\![\boldsymbol{c}^\star]\!]); \boldsymbol{c}, \boldsymbol{c}^\star)$$

where $\boldsymbol{c}, \boldsymbol{c}^\star \in \mathbb{F}^N$ and $\boldsymbol{c}^\star = F^{\mathsf{R}} \cdot \boldsymbol{c}$. Here F^{R} denotes the Fourier-like matrix defined as above.

Remark 1. *In addition, to achieve succinct verification in the resulting SNARK without committing to the structured matrix F^{R}, we introduce a new delegation protocol, described in Section A.1, that delegates the computation in Algorithm 2 to the prover. After integrating this delegation protocol, the prover time remains $O(N)$, while the verifier time and proof size become $O(\log^2 N)$.*

3.2 NTT Equality over Batched Polynomials

A key challenge of proving TFHE bootstrapping is that NTT operations are performed over a large number of distinct polynomials. This property makes straightforward batching-based proofs ineffective, as the verification cost grows with the number of polynomials. This issue is less prominent in the CNN inference considered by zkCNN [38], where Fourier transforms are typically applied to a small number of large polynomials. Thus, zkCNN does not address the many-polynomial batching problem that arises in TFHE bootstrapping.

We consider the problem of proving NTT equality over M distinct polynomials. A strawman approach is to execute Π_{NTT} for the single-polynomial NTT-equality relation $\mathcal{R}_{\mathsf{NTT}}$ independently for each polynomial. Although sum-check protocols across these executions can be batched to reduce the round complexity to $O(\log N)$, this approach still requires the verifier to query two oracles per execution of Π_{NTT}. As a result, the prover time is $O(MN)$, while the verifier performs $2M$ oracle queries, yielding a verifier time of $O(M + \log N)$.

This verifier cost is succinct only if M is small (e.g., $M = O(\log N)$). However, in the TFHE bootstrapping, the number of polynomials M can even exceed the size of each individual polynomial N (e.g., $M = 10{,}000$ and $N = 1024$). Moreover, when proving multiple FHE operations together with multiple bootstrappings, the verification cost becomes even more prohibitive, further limiting the scalability.

To address this issue, we introduce a new relation, denoted $\mathcal{R}_{\mathsf{B\text{-}NTT}}$ (Definition 2), together with a corresponding PIOP $\Pi_{\mathsf{B\text{-}NTT}}$ (Protocol 3.1), which proves NTT equality over M polynomials with verifier time independent of M.

Given M coefficient vectors $\boldsymbol{c}_i \in \mathbb{F}^N$ and their corresponding NTT vectors $\boldsymbol{c}_i^\star \in \mathbb{F}^N$ for $i \in [M]$, the relation $\mathcal{R}_{\mathsf{B\text{-}NTT}}$ is defined over two matrices $\mathbf{C} \in \mathbb{F}^{M \times N}$ and $\mathbf{C}^\star \in \mathbb{F}^{M \times N}$, whose rows consist of the coefficient vectors and their corresponding evaluation vectors, respectively. The relation of these matrices can be expressed as the matrix multiplication $\mathbf{C}^\star = \mathbf{C} \cdot \tilde{F}^{\mathsf{R}}$. As in the single-polynomial case, we rewrite this matrix multiplication as a multilinear polynomial identity:

$$\mathbf{C}^\star(y, x) = \sum_{k \in \{0,1\}^{\log N}} \mathbf{C}(k, x)\, \tilde{F}^{\mathsf{R}}(y, k), \tag{3}$$

Protocol 3.1. $\Pi_{\text{B-NTT}}$ *for* $\mathcal{R}_{\text{B-NTT}}$

1. $\mathcal{V}$ sends challenges $\boldsymbol{u} \leftarrow \mathbb{EF}^{\log N}$ and $\boldsymbol{v} \leftarrow \mathbb{EF}^{\log M}$
2. $\mathcal{P}$ computes $\mathsf{F}(x) \leftarrow \tilde{F}^{\mathsf{R}}(\boldsymbol{u}, x)$ using Algorithm 2 and computes the polynomial $\mathsf{C}(x) \leftarrow \mathbf{C}(x, \boldsymbol{v})$.
3. $\mathcal{P}$ defines the virtual oracle $\mathcal{G}(x) = \mathsf{F}(x) \cdot \mathsf{C}(x)$ and computes the value $T \leftarrow \mathbf{C}^{\star}(\boldsymbol{u}, \boldsymbol{v})$.
4. $\mathcal{V}$ queries the oracle $[\![\mathbf{C}^{\star}]\!]$ at the point $(\boldsymbol{u}, \boldsymbol{v})$.
5. $\mathcal{P}$ and $\mathcal{V}$ run Π_{Sum} for the $((T, [\![\mathcal{G}]\!]); \mathcal{G}) \in \mathcal{R}_{\mathsf{Sum}}$, where $\mathcal{V}$ answers any query to $\mathcal{G}(\boldsymbol{r})$ by making one query to $\mathsf{F}(\boldsymbol{u}, \boldsymbol{r})$ and one query to $\mathbf{C}(\boldsymbol{r}, \boldsymbol{v})$.
6. $\mathcal{V}$ accepts if the following checks are satisfied:
   - $\mathbf{C}^{\star}(\boldsymbol{u}, \boldsymbol{v}) = T$; and
   - the sum-check protocol Π_{Sum} accepts.

for all $y \in \{0,1\}^{\log N}$ and $x \in \{0,1\}^{\log M}$.

We observe that this problem can be viewed as verifying a matrix multiplication with a special Fourier structure. Although this relation can be verified using Thaler's general-purpose matrix-multiplication protocol [49, 50], such protocols do not exploit the Fourier structure of F^{R} and require committing to both input matrices, leading to asymptotically $O(MN + N^2)$ prover time. Instead, building on Algorithm 2, we design Protocol 3.1, which avoids committing to the structured F^{R} and achieves $O(MN)$ prover time. In addition, the verifier complexity of our protocol is sublinear in both the polynomial size N and the batch size M.

Def 2. *The Batched NTT-Equality relation* $\mathcal{R}_{\text{B-NTT}}$ *is the set of tuples*

$$(\mathbf{i}, \mathbf{x}, \mathbf{w}) = ((N, [\![F^{\mathsf{R}}]\!]), ([\![\mathbf{C}]\!], [\![\mathbf{C}^{\star}]\!]); \mathbf{C}, \mathbf{C}^{\star})$$

where $\mathbf{C}, \mathbf{C}^{\star} \in \mathbb{F}^{M \times N}$ *and* $\mathbf{C}^{\star} = \mathbf{C} \cdot \tilde{F}^{\mathsf{R}}$.

Protocol 3.1 achieves perfect completeness with soundness error $O(\log N + \log M)/|\mathbb{EF}|$. The proof follows the PIOP Π_{NTT} for the single NTT equality presented in Section B.2.

3.3 Batched Polynomial Multiplications

Although the computation circuit of a single bootstrapping is inherently sequential and admits little parallelism, it involves many repeated arithmetic operations on R_Q, i.e. (a sum of) polynomial multiplications in external product.

In this section, we present an efficient PIOP for the polynomial multiplication in batched form. Notably, the relation is defined solely in terms of coefficient-domain oracles, avoiding commitments to NTT-domain oracles in resulting SNARKs, which further improves the prover efficiency in practice.

Consider proving M polynomial multiplications over R_Q, i.e., $\boldsymbol{c}_i = \boldsymbol{a}_i \cdot \boldsymbol{b}_i$ for all $i \in [M]$. Working in the NTT domain reduces this task to two claims: (i) the NTT representations satisfy the element-wise product relation $\boldsymbol{c}_i^{\star} = \boldsymbol{a}_i^{\star} \circ \boldsymbol{b}_i^{\star}$, and (ii) NTT equality holds between each polynomial and its NTT representation. For small M, these claims can be verified using zero-check together with the single-polynomial NTT equality PIOP (Section 3.1); however, for large M, the verifier time grows linearly in M, undermining the succinctness.

To support a large M while maintaining succinct verifier time—particularly in applications such as TFHE bootstrapping—we adopt a batched formulation. Let $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C} \in \mathbb{F}^{M \times N}$ denote the coefficient matrices whose i-th rows are $\boldsymbol{a}_i$, $\boldsymbol{b}_i$, and $\boldsymbol{c}_i$, respectively. Similarly, let $\mathbf{A}^{\star}$, $\mathbf{B}^{\star}$, and $\mathbf{C}^{\star}$ denote their corresponding NTT matrices. Under this formulation, the two claims above are equivalently expressed as:

- $\mathbf{A}^{\star} = \mathbf{A} \cdot \tilde{F}^{\mathsf{R}}$, $\mathbf{B}^{\star} = \mathbf{B} \cdot \tilde{F}^{\mathsf{R}}$, and $\mathbf{C}^{\star} = \mathbf{C} \cdot \tilde{F}^{\mathsf{R}}$; and
- $\mathbf{C}^{\star} = \mathbf{A}^{\star} \circ \mathbf{B}^{\star}$.

The first claim can be proven using the batched NTT-equality PIOP $\Pi_{\text{B-NTT}}$ and the second claim is a Hadamard product relation over the NTT matrices, which can be proven using a zero-check protocol on the virtual oracle $\mathcal{G}(y,x) = \mathbf{A}^{\star}(y,x) \cdot \mathbf{B}^{\star}(y,x) - \mathbf{C}^{\star}(y,x)$. At the end of the protocol, the verifier queries $[\![\mathcal{G}]\!]$ at a random point $(\boldsymbol{u}, \boldsymbol{v}) \in \mathbb{EF}^{\log N} \times \mathbb{EF}^{\log M}$ by querying $[\![\mathbf{A}^{\star}]\!]$, $[\![\mathbf{B}^{\star}]\!]$, and $[\![\mathbf{C}^{\star}]\!]$.

An important observation is that the verifier's queries to the NTT-domain oracles $[\![\mathbf{A}^{\star}]\!]$, $[\![\mathbf{B}^{\star}]\!]$, and $[\![\mathbf{C}^{\star}]\!]$ at the random point $(\boldsymbol{u}, \boldsymbol{v})$ in the zero-check protocol Π_{Zero} need not be answered directly. Instead, PIOP $\Pi_{\text{B-NTT}}$ can be extended to evaluation proofs for NTT-domain oracles, with random challenges instantiated as evaluation points $(\boldsymbol{u}, \boldsymbol{v})$. This eliminates the need for the prover to commit to the NTT-domain oracles, which significantly improves the prover efficiency in practice.

Motivated by this observation, we define the batched polynomial multiplication relation $\mathcal{R}_{\text{B-Mult}}$ in Definition 3, where the public statement does *not* include the NTT-domain oracles. Due to space limitations, we defer the full protocol description of the PIOP $\Pi_{\text{B-Mult}}$ to the full version [37]. The resulting protocol achieves linear prover time $O(MN)$ and logarithmic verifier time $O(\log N + \log M)$, assuming that the prover has already computed and stored the NTT-domain matrices when evaluting the polynomial multiplications.

Def 3. *The Batched Polynomial Multiplication relation* $\mathcal{R}_{\text{B-Mult}}$ *is the set of tuples* $(\mathbf{i}, \mathbf{x}, \mathbf{w})$ *given by*

$$\big((M, N, [\![F^{\mathsf{R}}]\!]), ([\![\mathbf{A}]\!], [\![\mathbf{B}]\!], [\![\mathbf{C}]\!]); \mathbf{A}, \mathbf{B}, \mathbf{C}\big)$$

where $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{M \times N}$*, and letting* $\mathbf{A}^{\star} = \mathbf{A} \cdot \tilde{F}^{\mathsf{R}}$, $\mathbf{B}^{\star} = \mathbf{B} \cdot \tilde{F}^{\mathsf{R}}$, *and* $\mathbf{C}^{\star} = \mathbf{C} \cdot \tilde{F}^{\mathsf{R}}$*, it holds that* $\mathbf{C}^{\star} = \mathbf{A}^{\star} \circ \mathbf{B}^{\star}$.

This relation naturally generalizes to a broad class of batched arithmetic relations over R_Q, while involving only coefficient-domain oracles in the public statement. Our PIOP framework first proves the corresponding relation in the NTT domain via a zero-check protocol, reducing verification to evaluations of the NTT-domain matrices at a random point. The PIOP $\Pi_{\text{B-NTT}}$ then reduces these evaluations to coefficient-domain oracles.

3.4 Special Case: Multiplication by Monomials

In TFHE, there is a special operation referred to as the negacyclic rotation. Given a polynomial $\boldsymbol{c}(X) \in R_Q$ and an integer offset $a \in \mathbb{Z}$, a negacyclic rotation by a shifts the coefficient vector of $\boldsymbol{c}$ by a positions, with coefficients that wrap around negated. This operation is algebraically equivalent to multiplying $\boldsymbol{c}(X)$ by the monomial X^a in R_Q. However, in TFHE this operation is specified by the integer rotation offset a, rather than by an explicit coefficient representation of the corresponding monomial.

Motivated by this operation, we consider batched polynomial multiplications by monomials, where the monomial is succinctly specified by its degree and coefficient. Given M pairs of inputs $\big((a_i, b_i), \boldsymbol{g}_i\big)$ for $i \in [M]$, where $(a_i \in [N], b_i \in \mathbb{F})$ represents a monomial $b_i \cdot X^{a_i} \in R_Q$ and $\boldsymbol{g}_i \in R_Q$, the prover aims to convince the verifier that the products $\boldsymbol{h}_i = \boldsymbol{g}_i \cdot (b_i \cdot X^{a_i})$ are correctly computed in R_Q for all $i \in [M]$.

A strawman approach is to expand each monomial $b_i X^{a_i}$ into its full coefficient vector and form a matrix $\mathbf{C} \in \mathbb{F}^{M \times N}$. Similarly, we can form the coefficient matrices $\mathbf{G}$ and $\mathbf{H}$ corresponding to the polynomials $\boldsymbol{g}_i$ and $\boldsymbol{h}_i$, respectively. Under this representation, the relation reduces to a batched polynomial multiplication relation $\mathcal{R}_{\mathsf{B\text{-}Mult}}$. However, this reduction cannot be handled directly by the protocol $\Pi_{\mathsf{B\text{-}Mult}}$, since the coefficient matrix $\mathbf{C}$ corresponding to the monomials $b_i X^{a_i}$ is not committed as an oracle in the statement. Instead, $\mathbf{C}$ is implicitly specified by the succinct inputs $(\boldsymbol{a}, \boldsymbol{b})$, which actually form the sparse representation of $\mathbf{C}$.

Standard sparse polynomial commitment techniques [45, 46] show that a verifier query to a sparse matrix can be reduced to queries to its sparse representation, namely the vectors row, col, and val describing the locations and values of its non-zero entries. In our setting, the sparse representation of $\mathbf{C}$ is given by $\mathsf{col} = \boldsymbol{a}$ and $\mathsf{val} = \boldsymbol{b}$, while the row indices are fixed to $\mathsf{row} = [0, 1, \ldots, M-1]$. The multilinear extension of $\mathbf{C}$ evaluated at a point $(r_y, r_x) \in \mathbb{EF}^{\log N} \times \mathbb{EF}^{\log M}$ can therefore be expressed as $\tilde{\mathbf{C}}(r_y, r_x) =$

$$\sum_{k \in \{0,1\}^{\log M}} \mathsf{val}(k) \cdot \tilde{eq}(\mathsf{to\text{-}bits}(\mathsf{col}(k)), r_y) \cdot \tilde{eq}(k, r_x). \qquad (4)$$

following Claim 1 of [46], by setting $\mathsf{to\text{-}bits}(\mathsf{row}(k)) = k$ for all $k \in \{0,1\}^{\log M}$.

As a result, we can still apply the protocol $\Pi_{\mathsf{B\text{-}Mult}}$ to prove the batched polynomial multiplication by monomials relation, provided that the verifier queries to the oracle $[\![\mathbf{C}]\!]$ are reduced to queries to the sparse representation oracles $[\![\boldsymbol{a}]\!]$ and $[\![\boldsymbol{b}]\!]$ via the sparse matrix evaluation protocol.

We formalize the corresponding relation for batched polynomial multiplications by monomials as $\mathcal{R}_{\mathsf{B\text{-}Mono}}$ in Definition 4 and provide the PIOP $\Pi_{\mathsf{B\text{-}Mono}}$ in the full version [37].

Def 4. *The Batched Polynomial Multiplication by Monomials relation $\mathcal{R}_{\mathsf{B\text{-}Mono}}$ is the set of tuples $(\mathbf{i}, \mathbf{x}, \mathbf{w})$ given by*

$$\big((M, N, [\![F^{\mathsf{R}}]\!]), ([\![\boldsymbol{a}]\!], [\![\boldsymbol{b}]\!], [\![\mathbf{G}]\!], [\![\mathbf{H}]\!]); \boldsymbol{a}, \boldsymbol{b}, \mathbf{G}, \mathbf{H}\big),$$

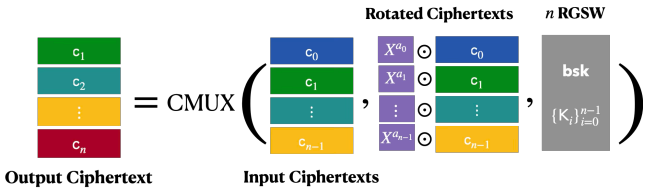


Figure 1: Batched instance of n CMUX Operations

where $\boldsymbol{a} \in [N]^M$ and $\boldsymbol{b} \in \mathbb{F}^M$. Let $\mathbf{C} \in \mathbb{F}^{M \times N}$ denote the (implicit) coefficient matrix whose i-th row is the coefficient vector of the monomial $b_i X^{a_i}$. The relation holds if

$$((M, N, [\![F^{\mathsf{R}}]\!]), ([\![\mathbf{C}]\!], [\![\mathbf{G}]\!], [\![\mathbf{H}]\!]); \mathbf{C}, \mathbf{G}, \mathbf{H}) \in \mathcal{R}_{\mathsf{B\text{-}Poly}}.$$

4 Arithmetic Relations in TFHE PBS

In this section, we focus on the arithmetic relations involved in the PBS procedure. The non-arithmetic relations, including digit decomposition as well as modulus switching, are addressed in Section 5.

4.1 Proving Batched CMUX in Blind Rotation

Blind rotation is the most computationally intensive step in TFHE PBS. It consists of a sequence of n ciphertext multiplexer (CMUX) operations: $(\mathsf{c}' - \mathsf{c}) \diamond \mathsf{K}_i + \mathsf{c}$, where c is the current RLWE ciphertext and $\mathsf{c}' = X^{a_i} \odot \mathsf{c}$ is its rotated counterpart specified by an offset a_i and K_i is an RGSW ciphertext. The rotation offset $a_i \in [2N]$ is produced by the preceding modulus switching step. This CMUX operation corresponds to the steps 7-9 in Algorithm 1.

Proving its correctness requires proving both the correctness of individual CMUX operation and their interconnections. In this section, we focus on the former and develop an efficient batched proof for the arithmetic relations within a large number of CMUX operations.

We consider each CMUX instance as taking its own RLWE ciphertext together with its rotated counterpart. Similarly, within each CMUX instance, the decomposition step and the subsequent polynomial multiplications in the external product can be handled instance by instance. Each CMUX instance produces its own RLWE ciphertext as output. We illustrate this conceptual view in Fig. 1.

This section focuses on the following arithmetic relations involved in CMUX. Non-arithmetic relations, namely decomposition, are addressed separately in Section 5.1.

1. negacyclic rotation in R_Q, which algebraically corresponds to multiplication by monomials; and
2. sums of polynomial multiplications that constitute the external product.

Negacyclic Rotation in R_Q. In each CMUX instance, the rotated RLWE ciphertext is obtained by applying a negacyclic rotation to both polynomial components of the input

ciphertext $\mathsf{c} = (\boldsymbol{a}, \boldsymbol{b})$. As shown in Section 3.4, this operation reduces to polynomial multiplication by a compactly represented monomial: writing the rotation offset $a \in [2N]$ as $a = kN + b$ with $b \in [N]$ and $k \in \{0,1\}$, the rotation corresponds to multiplication by $(1-2k) \cdot X^b$.

When proving n batched CMUX instances in blind rotation, each rotation offset $a_i \in [2N]$ can be written as $a_i = k_i N + b_i$ with $b_i \in [N]$ and $k_i \in \{0,1\}$, so that the corresponding negacyclic rotation reduces to multiplication by the monomial $(1-2k_i) \cdot X^{b_i}$. Correctness is established in two steps: (i) verifying that each pair (k_i, b_i) is correctly derived from a_i using an indexed lookup argument, since the relation between a_i and (k_i, b_i) is fixed and prescribed; and (ii) proving the resulting batched polynomial multiplications by monomials via our protocol $\Pi_{\text{B-Mono}}$.

Arithmetic Part of the External Product. Each CMUX instance also involves an external product ($\diamond$), as defined in Section 2. Ignoring the decomposition step, this operation can be written as a sum of polynomial multiplications $\boldsymbol{c} = \sum_{j=0}^{2\ell-1} \boldsymbol{a}_j \cdot \boldsymbol{b}_j$, where $\boldsymbol{a}_j$ arises from the decomposition of an RLWE ciphertext and $\boldsymbol{b}_j$ is a component of the RGSW ciphertext. This relation generalizes single polynomial multiplication and can be proved by extending the protocol $\Pi_{\text{B-Mult}}$ to handle batched sums of products across n CMUX instances.

The efficiency of proving batched CMUX is dominated by the cost of proving the arithmetic relations in the external products. This cost can be analyzed analogously to the PIOP $\Pi_{\text{B-Mult}}$ (see the full version [37]), yielding $O(N \cdot n \cdot \ell)$ prover time and $O(\log N + \log n)$ verifier time.

4.2 Proving the Wiring of Blind Rotation

In addition to proving the correctness of the CMUX operations, we also need to verify the proper wiring of the blind rotation, including (i) the inner wiring between consecutive CMUX operations; and (ii) the outer wiring between blind rotation and other stages of PBS. In this section, we show how to express these wiring constraints as arithmetic relations by a prescribed row permutation matrix and a difference matrix.

For simplicity, we consider only one polynomial component in the RLWE ciphertext; the other component is handled analogously. As conceptualized in Section 4.1, the input and output RLWE ciphertexts of blind rotation are batched into $n \times N$ matrices, and committed as separate oracles in the PIOP framework. As illustrated in Fig. 2, the wiring can be modeled by first applying a prescribed row permutation matrix P to the input RLWE ciphertext matrix, and then adding a difference matrix Z to obtain the output RLWE ciphertext matrix. Formally, this wiring is captured by the arithmetic relation

$$\mathsf{C}_{\text{out}} = P \cdot \mathsf{C}_{\text{in}} + \mathsf{Z}, \tag{5}$$

where C_{in} and C_{out} denote the input and output batched RLWE ciphertext matrices, respectively. As a result, the inner wiring between the consecutive CMUX operations is enforced by this prescribed permutation matrix P.

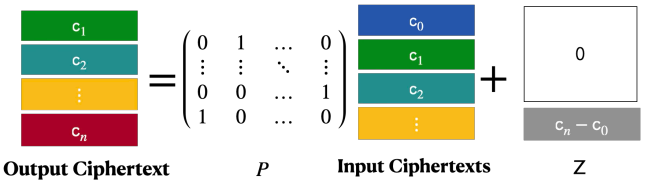


Figure 2: Wiring of CMUX modeled as a linear relation

Moreover, the outer wiring is also enforced by considering the difference matrix Z as a virtual oracle constructed from the initial RLWE ciphertext and the output of blind rotation. Let $\boldsymbol{a}$ be the initial RLWE ciphertext (one polynomial) and $\boldsymbol{b}$ the final output after blind rotation. The difference matrix $\mathsf{Z} \in \mathbb{F}^{n \times N}$ is defined to be zero except in last row, which equals the difference $\boldsymbol{b} - \boldsymbol{a}$. For simplicity, we assume n and N are powers of two (padding with dummy rows if needed). The multilinear extension of the difference matrix Z is

$$\begin{aligned}
\tilde{\mathsf{Z}}(y,x) &= \sum_{u \in \{0,1\}^{\log N}, v \in \{0,1\}^{\log n}} \tilde{eq}(y,u) \cdot \tilde{eq}(x,v) \cdot \mathsf{Z}_{v,u} && (6)\\
&= \sum_{u \in \{0,1\}^{\log N}} \tilde{eq}(y,u) \cdot \tilde{eq}(x, \mathbf{1}^{\log n}) \cdot (\boldsymbol{b}(u) - \boldsymbol{a}(u)) && (7)\\
&= \tilde{eq}(x, \mathbf{1}^{\log n}) \cdot \big(\tilde{\boldsymbol{b}}(y) - \tilde{\boldsymbol{a}}(y)\big). && (8)
\end{aligned}$$

Therefore, the queries to the difference matrix $[\![\mathsf{Z}]\!]$ can be reduced to queries to $[\![\boldsymbol{a}]\!]$ and $[\![\boldsymbol{b}]\!]$, associated with the initial and final RLWE ciphertexts. Consequently, the outer wiring is guaranteed by construction through the virtual oracle $[\![\mathsf{Z}]\!]$.

Finally, this arithmetic relation is written as:

$$\mathsf{C}_{\text{out}}(y,x) = \sum_{k \in \{0,1\}^{\log n}} P(k,x) \cdot \mathsf{C}_{\text{in}}(y,k) + \mathsf{Z}(y,x). \tag{9}$$

for all $x \in \{0,1\}^{\log N}$ and $y \in \{0,1\}^{\log n}$. With random challenges $(\boldsymbol{u}, \boldsymbol{v}) \in \mathbb{EF}^{\log N} \times \mathbb{EF}^{\log n}$, it suffices to prove:

$$T = \sum_{k \in \{0,1\}^{\log n}} P(k, \boldsymbol{v}) \cdot \mathsf{C}_{\text{in}}(\boldsymbol{u}, k). \tag{10}$$

where $T = \mathsf{C}_{\text{out}}(\boldsymbol{u}, \boldsymbol{v}) - \tilde{eq}(v, \mathbf{1}^{\log n}) \cdot (\tilde{\boldsymbol{b}}(u) - \tilde{\boldsymbol{a}}(u))$ is computed by querying the oracles $[\![\mathsf{C}_{\text{out}}]\!]$, $[\![\boldsymbol{a}]\!]$ and $[\![\boldsymbol{b}]\!]$. Then we can construct the virtual oracle $\mathcal{G}(k) = P(k,v) \cdot \mathsf{C}_{\text{in}}(u,k)$ and invoke the PIOP Π_{Sum} to prove that $(T, [\![\mathcal{G}]\!]; \mathcal{G}) \in \mathcal{R}_{\mathsf{Sum}}$. The permutation matrix P is public and prescribed, and can be committed in index, either as a standard polynomial or a sparse polynomial using protocols in the full version [37].

4.3 Proving Key Switching

The arithmetic structure of key switching can be viewed as a simpler variant of blind rotation. In a single PBS invocation, key switching involves only one ($\otimes$) multiplication between a polynomial and an RLev ciphertext. This operation

has the same algebraic form as an external product, namely, $\boldsymbol{c} = \sum_{j=0}^{\ell-1} \boldsymbol{a}_j \cdot \boldsymbol{b}_j$, where each $\boldsymbol{a}_j \in R_Q$ arises from the decomposition of a polynomial, and each $\boldsymbol{b}_j \in R_Q$ is a component polynomial of the RLev ciphertext. Consequently, the arithmetic correctness of a single key switching instance can be verified with the PIOP framework on R_Q with batch size one.

When multiple PBS invocations are considered, the associated key switching operations are independent across invocations. In this setting, we adopt the same batching strategy as for blind rotation and jointly prove all key switching operations as batched arithmetic relations, with the batch size equal to the number of PBS invocations.

4.4 Proving Sample Extraction

Sample extraction is the final stage of PBS, which extracts an LWE ciphertext encrypting the constant coefficient from an RLWE ciphertext $(\boldsymbol{a}, \boldsymbol{b}) \in R_Q$. Specifically, it constructs a valid LWE ciphertext $(\boldsymbol{a}', b') \in \mathbb{Z}_Q^{N+1}$ such that $(\boldsymbol{b} - \boldsymbol{a} \cdot \boldsymbol{s})_0 = b' - \langle \boldsymbol{a}', \boldsymbol{s} \rangle$. One valid instantiation sets $b' = b_0$ and $\boldsymbol{a}' = (a_0, -a_{N-1}, \ldots, -a_1)$, which is a signed permutation of the coefficients of $\boldsymbol{a}$. The correctness follows from the identity

$$a \cdot s = \sum_{i=0}^{N-1} a \cdot s_i X^i = \sum_{i=0}^{N-1} (X^i \cdot a) s_i, \tag{11}$$

where the i-th entry of $\boldsymbol{a}'$ equals the constant coefficient of $X^i \cdot \boldsymbol{a}$. Equivalently, this transformation can be written as a linear relation $\boldsymbol{a}' = Q \cdot \boldsymbol{a}$, where $Q \in \{-1, 0, 1\}^{N \times N}$ is a fixed signed permutation matrix. Concretely, $Q_{0,0} = 1$ and $Q_{i,N-i} = -1$ for $i = 1, \ldots, N-1$.

This linear relation can be verified by checking the corresponding multilinear equality $\tilde{\boldsymbol{a}}'(x) = \sum_{y \in \{0,1\}^{\log N}} \tilde{Q}(y, x) \cdot \tilde{\boldsymbol{a}}(y)$, which can be reduced to a sum-check. Note that the matrix Q is sparse and fixed, and can be committed as a sparse polynomial in the index of the protocol.

5 Non-Arithmetic Relations in TFHE PBS

In this section, we present PIOPs for the non-arithmetic relations that arise in TFHE programmable bootstrapping and are not covered in Section 4. These include modulus switching, and polynomial decomposition within blind rotation and key switching.

5.1 Decomposition and Range-Check

Both blind rotation and key switching require decomposing polynomials into base-B digit polynomials. Concretely, for a polynomial in R_Q with coefficient vector $\boldsymbol{a} \in \mathbb{F}^N$, a base-$B$ decomposition is performed element-wise, producing ℓ digit vectors $\boldsymbol{a}_0, \ldots, \boldsymbol{a}_{\ell-1} \in \mathbb{F}^N$.

The correctness of this decomposition is characterized by two conditions: (1) (*Range*) each entry of every digit vector $\boldsymbol{a}_j$ lies in $[0, B)$; and (2) (*Recombination*) the original vector can be recovered as $\boldsymbol{a} = \sum_{j=0}^{\ell-1} \boldsymbol{a}_j \cdot B^j$.

While the first condition can be verified directly using a lookup argument to check the range of each digit, the second condition is more subtle, since the recombination constraint must hold in the integer domain rather than merely in the field domain. Specifically, let k denote the bit-length of the prime modulus Q. A malicious prover could decompose a field element $a \in \mathbb{F}$ into digits lying in the larger range $[0, 2^k)$ instead of $[0, Q)$, such that the recombination of these digits equals $a + Q \in [0, 2^k)$. Therefore, we must enforce that the recombined value is strictly less than Q in the integer domain.

We adopt a decompose-then-lookup technique originally proposed in [33] for integer comparison, and instantiate the underlying lookup argument with LogUp [32]. The key idea is to reduce the comparison into a sequence of digit-wise comparison and equality checks, which can then be recursively combined into the final comparison result. For example, when comparing two base-B numbers with two digits, the comparison can be written as

$$\mathbf{1}\{a < c\} = \mathbf{1}\{a_1 < c_1\} + \mathbf{1}\{a_1 = c_1\} \cdot \mathbf{1}\{a_0 < c_0\}, \tag{12}$$

where $a = a_0 + a_1 \cdot B$ and $c = c_0 + c_1 \cdot B$.

Since this comparison technique operates directly on digit decompositions, it naturally generalizes to a range-check that simultaneously enforces the correctness of the decomposition in the integer domain. We formalize this range-check relation $\mathcal{R}_{\mathsf{Range}}$ in Definition 5 and further present a PIOP Π_{Range} for this relation in the full version [37].

Def 5. *The Range-Check relation $\mathcal{R}_{\mathsf{Range}}$ is the set of tuples $(\mathbf{i}, \mathbf{x}, \mathbf{w})$ given by*

$$\big((N, c, B, k), (\llbracket \boldsymbol{a} \rrbracket, \llbracket \boldsymbol{a}_0 \rrbracket, \ldots, \llbracket \boldsymbol{a}_{k-1} \rrbracket); \boldsymbol{a}, \boldsymbol{a}_0, \ldots, \boldsymbol{a}_{k-1}\big),$$

where $\boldsymbol{a}_0, \ldots, \boldsymbol{a}_{k-1} \in [B]^N$, and $\boldsymbol{a} = \sum_{j=0}^{k-1} \boldsymbol{a}_j \cdot B^j$ holds in the integer domain, and $a_i < c$ for all $i \in [N]$.

Our protocol in Section 4.1 proves only the arithmetic relations in blind rotation; the remaining components correspond to the polynomial decompositions inside the CMUX operations. These decompositions are independent across n CMUX instances and can therefore be verified in parallel. Concretely, this reduces to decomposing a coefficient matrix $\mathbf{A} \in \mathbb{F}^{n \times N}$ into ℓ digit matrices $\mathbf{A}_0, \ldots, \mathbf{A}_{\ell-1} \in \mathbb{F}^{n \times N}$. We then invoke the protocol Π_{Range} to verify the correctness of the decomposition by checking the instance

$$\big((n \cdot N, Q, B, \ell), (\llbracket \mathbf{A} \rrbracket, \llbracket \mathbf{A}_0 \rrbracket, \ldots, \llbracket \mathbf{A}_{\ell-1} \rrbracket); \mathbf{A}, \mathbf{A}_0, \ldots, \mathbf{A}_{\ell-1}\big).$$

The efficiency follows from our use of linear-time succinct lookup arguments [32], which yield a prover running time of $O(nN\ell)$ and a verifier running time of $O(\log n + \log N)$. A detailed analysis can be found in the full version [37].

Together with the protocol for proving the arithmetic relations in blind rotation, this establishes the overall correctness of the blind rotation.

Extension to Equality Check. The decompose-then-lookup technique used for range checks can be applied to equality checks. Given a vector $\boldsymbol{a} \in \mathbb{F}^N$ and a vector of claimed equality bits $\boldsymbol{e} \in \{0,1\}^N$ with respect to a public constant $c \in \mathbb{F}$, we can verify that $e_i = \mathbf{1}\{a_i = c\}$ for all $i \in [N]$.

In this work, we focus on the simplified equality check arising in modulus switching, where the public constant satisfies $c < \frac{Q-1}{2}$ and all entries of $\boldsymbol{a}$ are also bounded by $\frac{Q-1}{2}$. In this regime, it suffices to verify the decomposition purely in the field domain, since the recombined value of the digits is strictly less than Q. We formalize this as the equality relation $\mathcal{R}_{\mathsf{Eq}}$ in Definition 6 and the PIOP in the full version [37].

Def 6. *The Equality relation $\mathcal{R}_{\mathsf{Eq}}$ is the set of tuples $(\mathbf{i}, \mathbf{x}, \mathbf{w})$ given by*

$$\big((N, c, B, k), (\llbracket \boldsymbol{a} \rrbracket, \llbracket \boldsymbol{e} \rrbracket); \boldsymbol{a}, \boldsymbol{e}\big),$$

where $\boldsymbol{a} \in \mathbb{F}^N$ and $\boldsymbol{e} \in \{0,1\}^N$ satisfy $e_i = \mathbf{1}\{a_i = c\}$ for all $i \in [N]$. We assume that the public constant c and all entries of $\boldsymbol{a}$ are bounded by $\frac{Q-1}{2}$.

5.2 Modulus Switching

Modulus switching maps a field element $a \in \mathbb{F}$ of modulus Q to a ring element $b \in \mathbb{Z}_{2N}$ of modulus $2N$, which can be embedded into the multiplicative group $\langle x \rangle \in R_Q$ of size $2N$. This step ensures compatibility with subsequent polynomial operations in the blind rotation stage. Specifically, the modulus switching operation is defined as

$$b = \left\lfloor \frac{a \cdot 2N}{Q} \right\rceil \pmod{2N}, \tag{13}$$

which rounds the value $\frac{a \cdot 2N}{Q}$ to the nearest integer, followed by a reduction modulo $2N$. Notably, the rounded value $\lfloor \frac{a \cdot 2N}{Q} \rceil$ may wrap around modulo $2N$, potentially resulting in $b = 0$.

Assuming $4N \mid Q-1$, let $k = \frac{Q-1}{4N}$. Under this assumption, modulus switching admits a simple interval-based interpretation, as illustrated in Fig. 3. Specifically, it maps contiguous ranges of elements in $\mathbb{F}$ to the same element in $\mathbb{Z}_{2N}$. Ignoring the special case $a = k$, the field $\mathbb{F}$ can be partitioned into $2N$ disjoint segments, each of length $2k$. Specifically, for each $b' \in \{1, \ldots, 2N\}$, the segment

$$[(2b'-1) \cdot k + 1,\ (2b'+1) \cdot k]$$

is mapped to an output $b \in \mathbb{Z}_{2N}$ satisfying $b \equiv b' \pmod{2N}$. The exceptional point $a = k$ is mapped to $b = 0$. We formalize this observation in Theorem 1, with the parameter $q = 2N$.

Theorem 1. *Assume $2q \mid Q-1$, and $k = \frac{Q-1}{2q}$. For any $a \in \mathbb{F}_Q$ and $b \in \mathbb{Z}_q$, we have $b \equiv \lfloor \frac{a \cdot q}{Q} \rceil \pmod{q}$ if and only if either $a = k$ and $b = 0$, or there exists $b' \in \{1, \ldots, q\}$ such that $a \in [(2b'-1) \cdot k + 1, (2b'+1) \cdot k] \pmod{Q}$ and $b \equiv b' \pmod{q}$.*

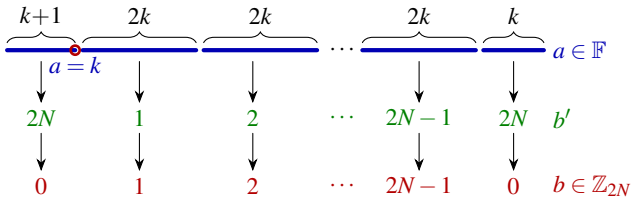


Figure 3: Illustration of modulus switching.

The proof of Theorem 1, together with a discussion of this assumption, is provided in Section C.

We show how the above characterization can be leveraged to verify the correctness of modulus switching. By introducing a witness b', the correctness of modulus switching can be verified using the following relations:

- $b \in [2N]$ and $b' \in \{1, \ldots, 2N\}$ such that $b' \equiv b \pmod{2N}$;
- $\mathbf{1}\{a \neq k\} = \mathbf{1}\{a - (2b'-1) \cdot k - 1 \in [0, 2k)\}$, where the right-hand side rewrites the interval condition in Theorem 1 as a range constraint;
- $\mathbf{1}\{a = k\} = \mathbf{1}\{b = 0\}$.

Considering typical FHE parameters, N is a small power of two (e.g., 1024 or 2048), and thus $k = \frac{Q-1}{4N}$ is generally large, with $2k < \frac{Q-1}{2}$. The first condition is captured by the indexed lookup relation $\mathcal{R}^*_{\mathsf{Lookup}}$, where the lookup table is fixed as $[2N, 1, \ldots, 2N-1]$. Here, b serves as the lookup index and b' is the corresponding lookup value. The second and the third condition are captured by the range-check relation $\mathcal{R}_{\mathsf{Range}}$ and the equality relation $\mathcal{R}_{\mathsf{Eq}}$, which are verified using the corresponding protocols in Section 5.1.

6 Implementation and Evaluation

We implemented HasteBoots [1], a verifiable FHE system, in approximately 56K lines of Rust code, including both the TFHE evaluation and the proof system for verifying TFHE correctness. All experiments were conducted on an Apple M4 (14 cores) machine with 24 GB of memory.

FHE Parameter Sets. We conduct experiments across three representative FHE parameter sets (Zama [51], BabyBear, and Goldilocks, as shown in Table 1). All three parameter sets support PBS over a 2-bit plaintext space and provide 128-bit estimated LWE security, matching the setting used in Zama [51]. We note one difference between the Zama parameters reported in [51] and our implementation. Zama [51] uses $B = 2^5$ and $\ell = 4$ with approximate decomposition. However, their circuit first computes the full decomposition and then keeps the higher-order digits as the approximate decomposition output. Thus, the decomposition witness size is essentially the same as that of full decomposition. Since directly expressing approximate decomposition as algebraic constraints is less straightforward in existing proof systems,

[1] https://doi.org/10.5281/zenodo.20275781

we use exact decomposition with the same base $B = 2^5$. This gives us a simpler and unambiguous relation while keeping the witness size comparable.

The BabyBear and Goldilocks parameter sets are selected using the LWE lattice estimator [1, 2] to target 128-bit estimated LWE security while being more SNARK-friendly than Zama's parameters. We note that changing FHE parameters can affect noise growth, correctness probability, and concrete PBS performance. A full production-level re-tuning and noise analysis is outside the scope of this work, and we refer readers to [5] for detailed TFHE parameter selections.

Parameter Set	Q	n	N	B	ℓ	Single-FHE Eval. Time (ms)
Zama	$2^{64}-2^{32}+1$	728	2^{10}	2^5	13	160.18
BabyBear	$2^{31}-2^{27}+1$	728	2^{10}	2^8	4	28.50
Goldilocks	$2^{64}-2^{32}+1$	728	2^{10}	2^8	8	107.96

Table 1: FHE parameter sets with 128-bit security, allowing for a plaintext space of size $t = 4$. Single-FHE evaluation time measures the time to perform one FHE primitive operation (including bootstrapping) without proof generation.

Experiment Setup. Our evaluation focuses on (i) the proving efficiency of individual FHE primitives, including prover time, verifier time, and proof size; (ii) the performance impact of different polynomial commitment schemes (PCS) and different FHE parameter choices; (iii) the scalability of our construction when batching multiple FHE operations; and (iv) the sensitivity of end-to-end performance to varying polynomial size N.

We instantiate our protocol with three PCS options (Brakedown [30], BaseFold [55], and Dory [35]), and evaluate both single-operation proof and batched proofs. More details on PCS are provided in the full version [37]. To support native arithmetic in our proof systems, the protocol operates on a 128-bit extension field $\mathbb{EF}$ constructed via an irreducible polynomial over the base field $\mathbb{F}_Q$, with the same modulus Q as in FHE parameters.

Overall PIOP Complexity Analysis. We estimate the circuit size of a single FHE primitive. The dominant cost arises from blind rotation, which consists of n sequential CMUX operations. Each CMUX operation involves 4ℓ polynomial multiplications over 6ℓ distinct polynomials. Accordingly, we approximate the circuit size of blind rotation by $O(\ell \cdot N \cdot n)$. Based on the complexity analyses in Sections 4.1 and 5.1, our overall construction achieves a linear-time prover of $O(\ell \cdot N \cdot n)$ with a succinct verifier running in $O(\log N + \log n)$ time. [2]

Single FHE Operation Proof. Table 2 reports the end-to-end performance of our protocol for proving a single FHE operation under different FHE parameter sets and PCS instantiations. Across all three parameter sets, we observe a tradeoff between prover efficiency, verifier cost, and proof size induced by the choice of PCS. Brakedown consistently achieves the lowest prover time, at the cost of substantially larger proof sizes and higher verifier overhead. Notably, for a single TFHE operation, Brakedown-based verification is slower than native FHE evaluation. In contrast, BaseFold yields extremely fast verification with moderate proof sizes, but incurs significantly higher prover time due to its heavier polynomial commitment and folding procedures. Dory provides a balanced trade-off, offering compact proofs and moderate verifier time while maintaining efficient prover time. Comparing across FHE parameter sets, we find that BabyBear and Goldilocks generally outperform Zama in prover time for all PCS choices, reflecting the advantage of SNARK-friendly parameters for arithmetic-intensive proof generation.

[2] This is the PIOP-level asymptotic complexity; the concrete SNARK performance also depends on the NTT delegation protocol and the chosen PCS backend.

Parameter Set	Proof System with PCS	Prover Time (s)	Verifier Time (ms)	Proof Size (MB)
Zama	[51]	25 **min**	4	0.20
Zama	H/Brakedown	8.45	427.33	181.55
Zama	H/Dory	12.36	118.12	0.22
Zama	H/BaseFold	153.03	6.29	4.93
BabyBear	H/Brakedown	3.51	224.24	135.45
BabyBear	H/Dory	5.47	109.35	0.24
BabyBear	H/BaseFold	45.53	4.04	4.65
Goldilocks	H/Brakedown	5.09	299.99	162.51
Goldilocks	H/Dory	10.10	118.08	0.21
Goldilocks	H/BaseFold	106.36	4.63	4.92

Table 2: End-to-end performance under different settings. H/Brakedown, H/Dory, and H/BaseFold denote HasteBoots instantiated with the Brakedown, Dory, and BaseFold PCS, respectively.

System	PIOP Protocol	PIOP Prover Time
Zama [51]	Plonk-style	10.2 min
HasteBoots	Sumcheck-based	5.77 s

Table 3: Comparison of PIOP-level prover time under the same Zama parameters, excluding PCS costs.

Comparison with Prior Work. We re-run the state-of-the-art [51] on an Apple M4 machine and compare the resulting performance with ours, as shown in Table 2. Zama [51] uses a Plonk-style PIOP protocol with a univariate FRI-based PCS, achieving a proving time of around half an hour.

However, as a general-purpose SNARK, Plonk has superlinear prover time in the size of the computation circuit, leading to inefficiencies when applied to the FHE circuits they construct. In particular, this incurs two layers of overhead for some FHE-specific operations compared to our approach. Specifically, one major bottleneck in [51] is the NTT operation, which is proved using an explicit NTT circuit of size $O(N \log N)$, resulting in a strictly quasi-linear-time prover with Plonk. In contrast, our protocol achieves linear-time proving by directly verifying the NTT relation between the inputs

Batch	HasteBoots w. Brakedown			HasteBoots w. Dory		
	Prover Time (s)	Verifier Time (ms)	Proof Size (MB)	Prover Time (s)	Verifier Time (ms)	Proof Size (MB)
1	3.51	224.24	135.45	5.47	109.35	0.24
2	5.35	287.37	169.92	8.94	118.17	0.24
4	10.2	365.68	196.85	15.47	117.89	0.25
8	20.75	521.07	250.26	29.06	125.85	0.27
16	49.41	749.82	293.29	60.90	126.00	0.28

Table 4: End-to-end performance under different batch sizes, using the BabyBear parameter set.

$\log N$	HasteBoots w. Brakedown			HasteBoots w. Dory		
	Prover Time (s)	Verifier Time (ms)	Proof Size (MB)	Prover Time (s)	Verifier Time (ms)	Proof Size (MB)
9	0.96	110.73	105.27	1.86	100.48	0.16
10	1.90	178.12	126.86	3.45	110.44	0.29
11	3.33	211.64	141.93	5.33	109.91	0.43
12	6.53	345.64	174.90	9.84	119.60	0.70
13	14.33	428.47	201.61	18.88	119.79	1.22
14	30.51	659.33	257.73	37.65	130.50	2.28

Table 5: End-to-end performance under varying polynomial size N. The experiments use the BabyBear field and fix $n = 2^9$, $B = 2^8$, and the remaining parameters.

and outputs, avoiding explicit circuit encoding.

To better understand the contributions of our specialized PIOP protocols, we isolate the PIOP prover time from PCS costs and compare it with the corresponding PIOP-level prover time in Zama, as shown in Table 3. Under the same parameters, Zama's PIOP-level prover time is 10.2 minutes (out of 25 minutes end-to-end), whereas ours is 5.77 seconds, more than a 100× improvement. The remaining PCS prover time in our end-to-end measurements is 2.68 s with Brakedown, 6.59 s with Dory, and 147.26 s with BaseFold. This breakdown shows that the prover-side speedup is not solely due to the choice of PCS: it also comes from our specialized PIOPs, which prove structured FHE relations directly and avoid explicit circuit encodings such as the NTT circuit used in prior work.

However, we emphasize that HasteBoots is not a strict improvement over Zama [51]; rather, it explores a different point in the prover–verifier–proof-size trade-off. Under the same Zama parameters, HasteBoots with Brakedown improves prover time by roughly 180×, but its verifier is about 100× slower and its proof size is about 900× larger. Using Dory reduces the proof size to 0.22 MB, comparable to Zama's 0.20 MB, but the verifier remains about 30× slower. This gap is largely due to Zama's use of recursion to compress verification and proof size. Our current implementation does not include recursive compression, but HasteBoots can also be extended with recursion, e.g., using recursion-friendly PCS backends or folding-based approaches. In the verifiable-FHE setting, however, this prover-side improvement is particularly important because proof generation is often the dominant bottleneck, while verification can be amortized by batching or reduced further through recursion.

zkVM-based approaches such as RISC0 [44] and SP1 [47] cannot be evaluated on local machines due to their substantial computational and memory requirements. Based on the reported results in [51], they require roughly 3 days and 1 day to prove a single FHE operation on an AWS `hpc7a.96xlarge` instance, respectively.

Batched FHE Operations Proof. We note that the benefit of verifiable FHE should be interpreted in the context of larger outsourced computations rather than a single isolated PBS. For a single TFHE bootstrapping operation, Brakedown-based verification can take longer than native FHE evaluation. Furthermore, a single PBS is only a low-level building block, and the prover cost of a TFHE program is directly tied to the number of PBS operations it requires. For example, using the 4-bit PBS parameters from the TFHE-rs handbook [54], a 64-bit integer multiplication requires 1772 PBS operations, corresponding to roughly 145 minutes of prover time with our current implementation, whereas a 16-bit integer multiplication requires 116 PBS operations, corresponding to roughly 10 minutes of prover time. Increasing the plaintext space of PBS reduces the number of PBS operations needed by higher-level programs, but also makes PBS substantially more expensive. In practice, small plaintext spaces such as 2-bit or 4-bit PBS are commonly used to balance the overall performance.

In our work, we evaluate the scalability of our construction by generating batched proofs for multiple 2-bit PBS operations in Table 4. Table 4 reports the performance of batched evaluation instantiated with the Brakedown and Dory PCS under the BabyBear parameter set. As the batch size increases from 1 to 16, prover time grows approximately linearly with the number of operations. In contrast, verifier time grows much more slowly, reflecting the advantage of batched verification for larger TFHE programs. To the best of our knowledge, existing SNARK-for-FHE systems do not provide a concrete construction nor an evaluation for batched FHE operation proofs. Notably, Zama [51] requires about half an hour to prove a single FHE operation, indicating that it does not scale to batched verification scenarios.

Support for Varying FHE Parameters. HasteBoots can be instantiated with different FHE parameter choices, which may adjust the PBS plaintext space, noise growth, correctness probability, and concrete performance. Among these parameters, the polynomial size N can vary widely across parameter choices, e.g., from 2^9 to 2^{14}, while the other parameters typically vary over a smaller range. To illustrate how performance changes with N, we run a controlled end-to-end experiment in Table 5, where we vary the polynomial size N while fixing the other parameters. The results show that prover time increases with the polynomial size, while verifier time grows much more slowly. Under the Dory instantiation, the proof size is more sensitive to N than in Table 4 because

Dory's PCS proof is already compact and the NTT delegation protocol becomes the dominant contributor to proof size.

Scope Beyond TFHE. HasteBoots is currently specialized to TFHE bootstrapping, especially CMUX gates, blind rotation, gadget decomposition, modulus switching, and negacyclic NTT operations. It is therefore not a drop-in proof system for BGV [9] or BFV [23] or CKKS [16]. Nevertheless, a main contribution underlying HasteBoots is a scalable proof framework for arithmetic relations over polynomial rings. Such relations also arise in other RLWE-based FHE schemes as well as lattice-based cryptographic systems [21]. We expect our approach of extracting and proving structured algebraic relations, and reducing non-algebraic checks to lookup relations, to be useful beyond TFHE. Extending this approach to other FHE schemes would require new relations tailored to their scheme-specific operations, such as relinearization, rotations, rescaling, and CKKS approximate arithmetic.

Acknowledgment

We thank anonymous reviewers and our shepherd for their constructive feedback. We also thank Tianyu Zhang for contributions to the early-stage implementation of this project.

Yu Yu is supported by the National Natural Science Foundation of China (Grant Nos. 62125204 and 92270201) and Innovation Program for Quantum Science and Technology (No. 2021ZD0302900). Yuncong Hu is supported by the National Natural Science Foundation of China (Grant Nos. 62502306), the ExploreX project of SJTU, and gifts/awards from Xiaomi.

Ethical Considerations

Stakeholder Identification and Impacts. Our work studies verifiable FHE and SNARK constructions and does not involve human subjects, physical experiments, or the collection of real-world private data.

Direct stakeholders include the research team and developers or organizations that implement or deploy our schemes in systems such as blockchain platforms, privacy-preserving computation, or verifiable computing services. These stakeholders may benefit from improved efficiency and stronger integrity guarantees, but may also face risks if the protocol assumptions, parameter choices, or limitations are misunderstood in deployment.

Indirect stakeholders include end users whose assets or data rely on such systems using verifiable FHE. They may benefit from reduced data exposure and verifiable outsourced computation, but could be harmed by immature or improperly reviewed deployments, potentially leading to financial loss, privacy breaches, or service disruptions. Broader affected parties may also face risks if privacy-preserving technologies are misused by illicit actors to conceal unlawful activities.

Benefits, Potential Harms, and Mitigations. Our work improves the efficiency of verifiable computation while preserving data privacy. To mitigate the risks to developers, deployers, and end users identified above, we document security assumptions, parameter choices, performance trade-offs, and known limitations, and we emphasize the need for careful implementation, auditing, and compliance with applicable laws and regulations before production use.

Publication Decision. We believe that the benefits of advancing efficient and verifiable FHE outweigh the potential risks when the schemes are deployed responsibly. We therefore consider publication appropriate and in line with established norms in cryptography research.

Open Science

Our code and experimental data are available at: https://doi.org/10.5281/zenodo.20275781.

References

[1] Martin Albrecht, Robert Player, and Sam Scott. Lattice estimator. https://github.com/malb/lattice-estimator, 2015. Accessed: 2026-01.

[2] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. Cryptology ePrint Archive, Report 2015/046, 2015. URL: https://eprint.iacr.org/2015/046.

[3] Diego F. Aranha, Anamaria Costache, Antonio Guimarães, and Eduardo Soria-Vazquez. HELIOPOLIS: Verifiable computation over homomorphically encrypted data from interactive oracle proofs is practical. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part V*, volume 15488 of *LNCS*, pages 302–334. Springer, Singapore, December 2024. doi:10.1007/978-981-96-0935-2_10.

[4] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast reed-solomon interactive oracle proofs of proximity. In Ioannis Chatzigiannakis, Christos Kaklamanis, Dániel Marx, and Donald Sannella, editors, *ICALP 2018*, volume 107 of *LIPIcs*, pages 14:1–14:17. Schloss Dagstuhl, July 2018. doi:10.4230/LIPIcs.ICALP.2018.14.

[5] Loris Bergerat, Anas Boudi, Quentin Bourgerie, Ilaria Chillotti, Damien Ligier, Jean-Baptiste Orfila, and Samuel Tap. Parameter optimization and larger precision for (T)FHE. *Journal of Cryptology*, 36(3):28, July 2023. doi:10.1007/s00145-023-09463-5.

[6] Alexandre Bois, Ignacio Cascudo, Dario Fiore, and Dongwoo Kim. Flexible and efficient verifiable computation on encrypted data. In Juan Garay, editor,

PKC 2021, Part II, volume 12711 of *LNCS*, pages 528–558. Springer, Cham, May 2021. doi:10.1007/978-3-030-75248-4_19.

[7] Jonathan Bootle, Alessandro Chiesa, Yuncong Hu, and Michele Orru. Gemini: Elastic snarks for diverse environments. In *EUROCRYPT*, 2022.

[8] Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical GapSVP. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 868–886. Springer, Berlin, Heidelberg, August 2012. doi:10.1007/978-3-642-32009-5_50.

[9] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (Leveled) fully homomorphic encryption without bootstrapping. In Shafi Goldwasser, editor, *ITCS 2012*, pages 309–325. ACM, January 2012. doi:10.1145/2090236.2090262.

[10] Zvika Brakerski and Vinod Vaikuntanathan. Fully homomorphic encryption from ring-LWE and security for key dependent messages. In Phillip Rogaway, editor, *CRYPTO 2011*, volume 6841 of *LNCS*, pages 505–524. Springer, Berlin, Heidelberg, August 2011. doi:10.1007/978-3-642-22792-9_29.

[11] Benedikt Bünz, Ben Fisch, and Alan Szepieniec. Transparent SNARKs from DARK compilers. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 677–706. Springer, Cham, May 2020. doi:10.1007/978-3-030-45721-1_24.

[12] Ignacio Cascudo, Anamaria Costache, Daniele Cozzo, Dario Fiore, Antonio Guimarães, and Eduardo Soria-Vazquez. Verifiable computation for approximate homomorphic encryption schemes. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VII*, volume 16006 of *LNCS*, pages 643–677. Springer, Cham, August 2025. doi:10.1007/978-3-032-01907-3_21.

[13] Bhuvnesh Chaturvedi, Anirban Chakraborty, Ayantika Chatterjee, and Debdeep Mukhopadhyay. vr^{2}FHE-securing FHE from reaction-based key recovery attacks. Cryptology ePrint Archive, Report 2023/561, 2023. URL: https://eprint.iacr.org/2023/561.

[14] Bhuvnesh Chaturvedi, Anirban Chakraborty, Ayantika Chatterjee, and Debdeep Mukhopadhyay. "Ask and thou shall receive": Reaction-based full key recovery attacks on FHE. In Joaquin Garcia-Alfaro, Rafał Kozik, Michał Choraś, and Sokratis Katsikas, editors, *ESORICS 2024, Part IV*, volume 14985 of *LNCS*, pages 457–477. Springer, Cham, September 2024. doi:10.1007/978-3-031-70903-6_23.

[15] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. HyperPlonk: Plonk with linear-time prover and high-degree custom gates. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part II*, volume 14005 of *LNCS*, pages 499–530. Springer, Cham, April 2023. doi:10.1007/978-3-031-30617-4_17.

[16] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yong Soo Song. Homomorphic encryption for arithmetic of approximate numbers. In Tsuyoshi Takagi and Thomas Peyrin, editors, *ASIACRYPT 2017, Part I*, volume 10624 of *LNCS*, pages 409–437. Springer, Cham, December 2017. doi:10.1007/978-3-319-70694-8_15.

[17] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Psi Vesely, and Nicholas P. Ward. Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 738–768. Springer, Cham, May 2020. doi:10.1007/978-3-030-45721-1_26.

[18] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. TFHE: Fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, January 2020. doi:10.1007/s00145-019-09319-x.

[19] Ilaria Chillotti, Nicolas Gama, and Louis Goubin. Attacking FHE-based applications by software fault injections. Cryptology ePrint Archive, Report 2016/1164, 2016. URL: https://eprint.iacr.org/2016/1164.

[20] Ilaria Chillotti, Damien Ligier, Jean-Baptiste Orfila, and Samuel Tap. Improved programmable bootstrapping with larger precision and efficient arithmetic circuits for TFHE. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part III*, volume 13092 of *LNCS*, pages 670–699. Springer, Cham, December 2021. doi:10.1007/978-3-030-92078-4_23.

[21] Thinh Dang, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, Dustin Moody, Rene Peralta, Ray Perlner, Angela Robinson, et al. Module-lattice-based digital signature standard. 2024.

[22] Léo Ducas and Daniele Micciancio. FHEW: Bootstrapping homomorphic encryption in less than a second. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part I*, volume 9056 of *LNCS*, pages 617–640. Springer, Berlin, Heidelberg, April 2015. doi:10.1007/978-3-662-46800-5_24.

[23] Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. Cryptology

ePrint Archive, Report 2012/144, 2012. URL: https://eprint.iacr.org/2012/144.

[24] Dario Fiore, Anca Nitulescu, and David Pointcheval. Boosting verifiable computation on encrypted data. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020, Part II*, volume 12111 of *LNCS*, pages 124–154. Springer, Cham, May 2020. doi:10.1007/978-3-030-45388-6_5.

[25] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Report 2019/953, 2019. URL: https://eprint.iacr.org/2019/953.

[26] Mariana Gama, Emad Heydari Beni, Jiayi Kang, Jannik Spiessens, and Frederik Vercauteren. Blind zkSNARKs for private proof delegation and verifiable computation over encrypted data. Cryptology ePrint Archive, Report 2024/1684, 2024. URL: https://eprint.iacr.org/2024/1684.

[27] Chaya Ganesh, Anca Nitulescu, and Eduardo Soria-Vazquez. Rinocchio: SNARKs for ring arithmetic. *Journal of Cryptology*, 36(4):41, October 2023. doi:10.1007/s00145-023-09481-3.

[28] Sanjam Garg, Aarushi Goel, and Mingyuan Wang. How to prove statements obliviously? In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part X*, volume 14929 of *LNCS*, pages 449–487. Springer, Cham, August 2024. doi:10.1007/978-3-031-68403-6_14.

[29] Craig Gentry. Fully homomorphic encryption using ideal lattices. In Michael Mitzenmacher, editor, *41st ACM STOC*, pages 169–178. ACM Press, May / June 2009. doi:10.1145/1536414.1536440.

[30] Alexander Golovnev, Jonathan Lee, Srinath T. V. Setty, Justin Thaler, and Riad S. Wahby. Brakedown: Linear-time and field-agnostic SNARKs for R1CS. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part II*, volume 14082 of *LNCS*, pages 193–226. Springer, Cham, August 2023. doi:10.1007/978-3-031-38545-2_7.

[31] Jens Groth. On the size of pairing-based non-interactive arguments. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 305–326. Springer, Berlin, Heidelberg, May 2016. doi:10.1007/978-3-662-49896-5_11.

[32] Ulrich Haböck. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Report 2022/1530, 2022. URL: https://eprint.iacr.org/2022/1530.

[33] Meng Hao, Hanxiao Chen, Hongwei Li, Chenkai Weng, Yuan Zhang, Haomiao Yang, and Tianwei Zhang. Scalable zero-knowledge proofs for non-linear functions in machine learning. In Davide Balzarotti and Wenyuan Xu, editors, *USENIX Security 2024*. USENIX Association, August 2024. URL: https://www.usenix.org/conference/usenixsecurity24/presentation/hao-meng-scalable.

[34] Christian Knabenhans, Alexander Viand, Antonio Merino-Gallardo, and Anwar Hithnawi. vfhe: Verifiable fully homomorphic encryption. In *Proceedings of the 12th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, WAHC '24, page 11–22, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3689945.3694806.

[35] Jonathan Lee. Dory: Efficient, transparent arguments for generalised inner products and polynomial commitments. In *Theory of cryptography conference*, 2021.

[36] Zengpeng Li, Steven D. Galbraith, and Chunguang Ma. Preventing adaptive key recovery attacks on the GSW levelled homomorphic encryption scheme. In Liqun Chen and Jinguang Han, editors, *ProvSec 2016*, volume 10005 of *LNCS*, pages 373–383. Springer, Cham, November 2016. doi:10.1007/978-3-319-47422-9_22.

[37] Fengrun Liu, Haofei Liang, Xiang Xie, Yu Yu, Wenting Zheng, and Yuncong Hu. HasteBoots: Proving FHE bootstrapping in seconds. Cryptology ePrint Archive, Paper 2025/261, 2025. URL: https://eprint.iacr.org/2025/261.

[38] Tianyi Liu, Xiang Xie, and Yupeng Zhang. zkCNN: Zero knowledge proofs for convolutional neural network predictions and accuracy. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 2968–2985. ACM Press, November 2021. doi:10.1145/3460120.3485379.

[39] Carsten Lund, Lance Fortnow, Howard J. Karloff, and Noam Nisan. Algebraic methods for interactive proof systems. In *31st FOCS*, pages 2–10. IEEE Computer Society Press, October 1990. doi:10.1109/FSCS.1990.89518.

[40] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In Henri Gilbert, editor, *EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 1–23. Springer, Berlin, Heidelberg, May / June 2010. doi:10.1007/978-3-642-13190-5_1.

[41] Daniele Micciancio and Yuriy Polyakov. Bootstrapping in FHEW-like cryptosystems. Cryptology ePrint

Archive, Report 2020/086, 2020. URL: https://eprint.iacr.org/2020/086.

[42] Kabir Peshawaria, Zeyu Liu, Ben Fisch, and Eran Tromer. Laminate: Succinct SIMD-friendly verifiable FHE. Cryptology ePrint Archive, Paper 2025/2285, 2025. URL: https://eprint.iacr.org/2025/2285.

[43] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In Harold N. Gabow and Ronald Fagin, editors, *37th ACM STOC*, pages 84–93. ACM Press, May 2005. doi:10.1145/1060590.1060603.

[44] RISC Zero. Risc zero: A zero-knowledge virtual machine. https://risczero.com/, 2023. Project website.

[45] Srinath Setty. Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part III*, volume 12172 of *LNCS*, pages 704–737. Springer, Cham, August 2020. doi:10.1007/978-3-030-56877-1_25.

[46] Srinath T. V. Setty, Justin Thaler, and Riad S. Wahby. Unlocking the lookup singularity with Lasso. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part VI*, volume 14656 of *LNCS*, pages 180–209. Springer, Cham, May 2024. doi:10.1007/978-3-031-58751-1_7.

[47] Succinct Labs. Sp1: A high-performance zero-knowledge virtual machine. https://github.com/succinctlabs/sp1, 2023. GitHub repository.

[48] Polygon Zero Team. Plonky2. https://github.com/0xPolygonZero/plonky2, 2022. Accessed: 2025-04-13.

[49] Justin Thaler. Time-optimal interactive proofs for circuit evaluation. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part II*, volume 8043 of *LNCS*, pages 71–89. Springer, Berlin, Heidelberg, August 2013. doi:10.1007/978-3-642-40084-1_5.

[50] Justin Thaler et al. Proofs, arguments, and zero-knowledge. *Foundations and Trends® in Privacy and Security*, 4(2–4):117–660, 2022.

[51] Louis Tremblay Thibault and Michael Walter. Towards verifiable FHE in practice: Proving correct execution of TFHE's bootstrapping using plonky2. In Chun-Ying Huang, Jyh-Cheng Chen, Shiuh-Pyng Shieh, David Lie, and Véronique Cortier, editors, *ACM CCS 2025*, pages 1113–1126. ACM Press, October 2025. doi:10.1145/3719027.3765097.

[52] Alexander Viand, Christian Knabenhans, and Anwar Hithnawi. Verifiable fully homomorphic encryption, 2023. URL: https://arxiv.org/abs/2301.07041, arXiv:2301.07041.

[53] Tiancheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. Libra: Succinct zero-knowledge proofs with optimal prover computation. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 733–764. Springer, Cham, August 2019. doi:10.1007/978-3-030-26954-8_24.

[54] Zama. TFHE-rs: A (Practical) Handbook. https://github.com/zama-ai/tfhe-rs-handbook/blob/main/tfhe-rs-handbook.pdf, 2025. First edition. Accessed: 2026-05-17.

[55] Hadas Zeilberger, Binyi Chen, and Ben Fisch. Basefold: efficient field-agnostic polynomial commitment schemes from foldable codes. In *Annual International Cryptology Conference*, pages 138–169. Springer, 2024.

[56] Xinxuan Zhang, Ruida Wang, Zeyu Liu, Binwu Xiang, Yi Deng, Ben Fisch, and Xianhui Lu. Phalanx: An FHE-friendly SNARK for verifiable computation on encrypted data. In Chun-Ying Huang, Jyh-Cheng Chen, Shiuh-Pyng Shieh, David Lie, and Véronique Cortier, editors, *ACM CCS 2025*, pages 4035–4048. ACM Press, October 2025. doi:10.1145/3719027.3765226.

[57] Zhenfei Zhang, Thomas Plantard, and Willy Susilo. Reaction attack on outsourced computing with fully homomorphic encryption schemes. In Howon Kim, editor, *ICISC 11*, volume 7259 of *LNCS*, pages 419–436. Springer, Berlin, Heidelberg, November / December 2012. doi:10.1007/978-3-642-31912-9_28.

A Additional Details for NTT Equality

A.1 Idea of Delegation Protocol

The evaluation for $\tilde{F}^{\mathsf{R}}(u,v)$ can be delegated to the prover through a sequence of sumcheck protocol, following the same Algorithm 2 to compute $A_{F^{\mathsf{R}}}$. We abuse the notation and use $A^{(i)}_{F^{\mathsf{R}}} : \{0,1\}^{i+1} \to \mathbb{EF}$ as in [38] to denote the array $A_{F^{\mathsf{R}}}$ in the i-th round for $i = 0, \ldots, \log N - 1$. Then $\tilde{F}^{\mathsf{R}}(u,v) = \tilde{A}^{(\log N-1)}_{F^{\mathsf{R}}}(v)$, and we can write $A^{(i)}_{F^{\mathsf{R}}}(\cdot)$ as an equation of $A^{(i-1)}_{F^{\mathsf{R}}}(\cdot)$:

$$A^{(i)}_{F^{\mathsf{R}}}(x,b) = A^{(i-1)}_{F^{\mathsf{R}}}(x)\left((1-u_i) + u_i \cdot \omega_{i+1}(x,b)\right) \cdot \omega^{2^i \cdot b}$$

for all $x \in \{0,1\}^i, b \in \{0,1\}$, where $\omega_{i+1}(x,b) = \omega^j_{2^{i+1}}$ with $j = \mathcal{X} + 2^i \cdot b$. Note that the right-hand side of the above equation is not multilinear in either x or b, so we introduce

a $(i+1)$-variate identity function to obtain the following multilinear function

$$\tilde{A}^{(i)}_{F^{\mathsf{R}}}(x,b) = \sum_{z\in\{0,1\}^i}\sum_{s\in\{0,1\}} \tilde{eq}((x,b),(z,s))\cdot\tilde{A}^{(i-1)}_{F^{\mathsf{R}}}(z) \cdot((1-u_i)+u_i\cdot\tilde{\omega}_{i+1}(z,s))\cdot\omega^{2^i\cdot s}$$

that holds for all $x\in\mathbb{EF}^i, b\in\mathbb{EF}$.

Starting from $\tilde{F}^{\mathsf{R}}(u,v)=\tilde{A}^{(\log N-1)}_{F^{\mathsf{R}}}(v)$, the verifier and the prover can reduce its correctness to the evaluation of $\tilde{A}^{(i)}_{F^{\mathsf{R}}}(\cdot)$ at a random point through a sumcheck protocol for $i=\log N-1,\ldots,0$. In the last round as defined in the Step 1 of Algorithm 2, $\tilde{A}^{(0)}_{F^{\mathsf{R}}}(\cdot)$ is simply the constant 1. At the end of each sumcheck protocol, the verifier has to evaluate $\tilde{eq}(\cdot)$ and $\tilde{\omega}_{i+1}(\cdot)$ at a random point to obtain $\tilde{A}^{(i-1)}_{F^{\mathsf{R}}}$ at the random point for the next sumcheck protocol. By the closed-form definition of multilinear extension, we have $\tilde{\omega}_{i+1}(r)=\sum_{x\in\{0,1\}^{i+1}}\tilde{eq}(r,x)\,\omega^j_{2^{i+1}}$ for $j=\sum_{k=0}^{i+1}2^k\cdot x_k$, which equals to $\prod_{k=0}^{i+1}(1-r_k+r_k\cdot\omega^{2^k}_{2^{i+1}})$.

A.2 Full Derivation of Eq. (2)

$$\tilde{F}^{\mathsf{R}}(u,x) = \sum_{y\in\{0,1\}^{\log N}}\tilde{eq}(u,y)\,\tilde{F}^{\mathsf{R}}(y,x) \tag{14}$$

$$= \sum_{y\in\{0,1\}^{\log N}}\tilde{eq}(u,y)\,\omega^{(2\gamma^{\mathsf{R}}+1)\cdot\mathcal{X}} \tag{15}$$

$$= \omega^{\mathcal{X}}\cdot\sum_{y\in\{0,1\}^{\log N}}\tilde{eq}(u,y)\,\omega^{\mathcal{X}\cdot 2\gamma^{\mathsf{R}}} \tag{16}$$

$$= \omega^{\mathcal{X}}\cdot\sum_{y\in\{0,1\}^{\log N}}\tilde{eq}(u,y)\,\omega^{\mathcal{X}\cdot(\sum_{i=0}^{\log N-1}2^{\log N-i}\cdot y_i)} \tag{17}$$

$$= \omega^{\mathcal{X}}\cdot\sum_{y\in\{0,1\}^{\log N}}\prod_{j=0}^{\log N-1}((1-u_j)(1-y_j)+u_jy_j) \cdot\prod_{i=0}^{\log N-1}\left(\omega^{2^{\log N-i}}\right)^{\mathcal{X}\cdot y_i} \tag{18}$$

$$= \omega^{\mathcal{X}}\cdot\sum_{y\in\{0,1\}^{\log N}}\prod_{i=0}^{\log N-1}((1-u_i)(1-y_i)+u_iy_i)\cdot\omega^{\mathcal{X}\cdot y_i}_{2^{i+1}} \tag{19}$$

$$= \omega^{\mathcal{X}}\cdot\prod_{i=0}^{\log N-1}\sum_{y_i\in\{0,1\}}((1-u_i)(1-y_i)+u_iy_i)\cdot\omega^{\mathcal{X}\cdot y_i}_{2^{i+1}} \tag{20}$$

$$= \omega^{\mathcal{X}}\cdot\prod_{i=0}^{\log N-1}(1-u_i+u_i\cdot\omega^{\mathcal{X}}_{2^{i+1}}) \tag{21}$$

$$= \omega^{\sum_{j=0}^{\log N-1}2^j\cdot x_j}\cdot\prod_{i=0}^{\log N-1}(1-u_i+u_i\cdot\omega^{\mathcal{X}}_{2^{i+1}}) \tag{22}$$

$$= \prod_{i=0}^{\log N-1}(1-u_i+u_i\cdot\omega^{\mathcal{X}}_{2^{i+1}})\cdot\omega^{2^i\cdot x_i} \tag{23}$$

Protocol B.1. Π_{Sum} *for* $\mathcal{R}_{Sum}$

1. For $j=1$ to μ:
   1.1 $\mathcal{P}$ computes and sends $g_j(X)\leftarrow\sum_{\boldsymbol{i}\in\{0,1\}^{\mu-j}}p(r_1,\ldots,r_{j-1},X,\boldsymbol{i})$
   1.2 $\mathcal{V}$ sends a random challenge $r_j\leftarrow\mathbb{EF}$.
2. $\mathcal{V}$ queries $[\![p]\!]$ on $(r_1,\ldots,r_\mu)$
3. $\mathcal{V}$ accepts if the following checks are satisfied:
   - $g_1(0)+g_1(1)=T$.
   - $g_j(r_{j+1})=g_{j+1}(0)+g_{j+1}(1)$ for $j=1,\ldots,\mu-1$.
   - $g_\mu(r_\mu)=p(r_1,\ldots,r_\mu)$.

B PIOP for NTT Equality

This appendix presents the NTT-equality PIOP used in Section 3.1. Since the protocol relies on the standard sum-check protocol, we first recall the sum-check relation and protocol for completeness. Other PIOP constructions are deferred to the full version [37].

B.1 Sum-check Protocol

We consider running sum-check protocols over the extension field $\mathbb{EF}$.

Def 7. *The sum-check relation $\mathcal{R}_{sum}$ is the set of tuples $(\mathbf{i},\mathbf{x},\mathbf{w})=(\perp,(T,[\![p]\!]);p)$ such that $T=\sum_{x\in\{0,1\}^\mu p(x)}$ where p is a μ-varitat polynomial over the extension field $\mathbb{EF}$ with degree at most d in each variable.*

Theorem 2. *The PIOP Π_{Sum} for $\mathcal{R}_{Sum}$ described in Protocol B.1 has perfect completeness and δ-knowledge soundness error with $\delta=\mu d/|\mathbb{EF}|$. The prover runs in time $O(2^\mu)$, while the verifier runs in time $O(\mu)$.*

Proof. It follows from the Proposition 4.1 of [50]. □

B.2 PIOP for $\mathcal{R}_{\mathsf{NTT}}$

Theorem 3. *The PIOP Π_{NTT} for the relation $\mathcal{R}_{NTT}$ (Definition 1) described in Protocol B.2 has perfect completeness and δ-knowledge soundness error with $\delta=O(\log N)/|\mathbb{EF}|$. The prover runs in time $O(N)$, while the verifier runs in time $O(\log N)$ with $O(\log N)$ communication complexity.*

Proof. The completeness follows directly from the definition of the relation $\mathcal{R}_{\mathsf{NTT}}$. Suppose $((N,[\![F^{\mathsf{R}}]\!]),([\![\boldsymbol{c},\boldsymbol{c}^\star]\!]);\boldsymbol{c},\boldsymbol{c}^\star)\notin\mathcal{R}_{\mathsf{NTT}}$, then we have that $\exists y$ such that $\boldsymbol{c}^\star(y)\neq\sum_{x\in\{0,1\}^{\log N}}\boldsymbol{c}(x)F^{\mathsf{R}}(y,x)$. Hence, the multilinear polynomial $\tilde{\boldsymbol{c}}(y)$ is not equal to $\sum_{x\in\{0,1\}^{\log N}}\boldsymbol{c}(x)\tilde{F}^{\mathsf{R}}(y,x)$, which is also multilinear polynomial defined over y, since $\tilde{F}^{\mathsf{R}}(y,x)$ is defined as the multilinear

Protocol B.2. Π_{NTT} *for* $\mathcal{R}_{NTT}$

1. $\mathcal{V}$ sends a random challenge $\boldsymbol{u} \in \mathbb{EF}^{\log N}$
2. $\mathcal{P}$ computes the polynomial $\mathsf{F}(x) \leftarrow \tilde{F}^{\mathsf{R}}(\boldsymbol{u}, x)$ using the Algorithm 2.
3. $\mathcal{P}$ defines the virtual oracle $\mathcal{G}(x) = \mathsf{F}(x) \cdot \boldsymbol{c}(x)$ and computes $T \leftarrow \boldsymbol{c}^{\star}(\boldsymbol{u})$.
4. $\mathcal{V}$ queries $[\![\boldsymbol{c}^{\star}]\!]$ on $\boldsymbol{u}$.
5. $\mathcal{P}$ and $\mathcal{V}$ run the sum-check protocol Π_{Sum} for $((T, [\![\mathcal{G}]\!]); \mathcal{G}) \in \mathcal{R}_{\mathsf{Sum}}$ where $\mathcal{V}$ answers any query to $\mathcal{G}(\boldsymbol{r})$ by making one to $\mathsf{F}(\boldsymbol{r})$ and $\boldsymbol{c}(\boldsymbol{r})$, respectively.
6. $\mathcal{V}$ accepts if the following checks are satisfied:
   - $\boldsymbol{c}^{\star}(\boldsymbol{u}) = T$.
   - The sum-check protocol Π_{Sum} accepts.

extension in the variables y and x. By the Schwartz-Zippel lemma, the probability that $\tilde{\boldsymbol{c}}(\boldsymbol{u}) = \sum_{x \in \{0,1\}^{\log N}} \boldsymbol{c}(x) \tilde{F}^{\mathsf{R}}(\boldsymbol{u}, x)$ is at most $\log N / |\mathbb{EF}|$ for a random challenge $\boldsymbol{u} \leftarrow \mathbb{EF}^{\log N}$. On the other hand, by the soundness of the sum-check protocol PIOP (Theorem 2), it remains to justify the degree bound of the virtual oracle $\mathcal{G}(x) = \boldsymbol{c}(x) \tilde{F}^{\mathsf{R}}(\boldsymbol{u}, x)$. After the verifier samples and fixes $\boldsymbol{u}$, the polynomial $\tilde{F}^{\mathsf{R}}(\boldsymbol{u}, x)$ remains multilinear in the varibales x. Consequently, $\mathcal{G}(x)$ has individual degree at most 2 in each variable x_i, so the soundness error from the sum-check protocol is at most $2 \log N / |\mathbb{EF}|$. Combining this with the Schwartz–Zippel error above, the probability that the verifier accepts a tuple $((N, [\![F^{\mathsf{R}}]\!]), ([\![\boldsymbol{c}, \boldsymbol{c}^{\star}]\!]); \boldsymbol{c}, \boldsymbol{c}^{\star}) \notin \mathcal{R}_{\mathsf{NTT}}$ is at most $3 \log N / |\mathbb{EF}|$.

The prover time is dominated by the computation of the polynomial $\mathsf{F}(x) = \tilde{F}^{\mathsf{R}}(\boldsymbol{u}, x)$ for all $x \in \{0,1\}^{\log N}$. Although the direct expression $\tilde{F}^{\mathsf{R}}(\boldsymbol{u}, x) = \sum_{y \in \{0,1\}^{\log N}} \tilde{eq}(u, y) \tilde{F}^{\mathsf{R}}(y, x)$ suggests a higher cost, the reformulation $\prod_{i=0}^{\log N - 1} \left(1 - u_i + u_i \cdot \omega_{2^{i+1}}^{x}\right) \cdot \omega^{2^i \cdot x_i}$ as derived in Section A.2, preserves the same function while enabling Algorithm 2 to compute all required values in $O(N)$ time. The verifier runs in time $O(\log N)$ and the total communication complexity is $O(\log N)$, which follows from the efficiency of the sum-check protocol. □

C Additional Details for Modulus Switching

C.1 Proof of Theorem 1

Proof. Now, consider $a \in \mathbb{F} \setminus \{k\}$. Using the identity $kq = \frac{1}{2}(Q-1)$, we analyze the value of $\lfloor \frac{a \cdot q}{Q} \rceil$ by distinguishing the value of b'.

Case 1: $b' \in \{1, \ldots, q-1\}$**.** In this case, we have

$$a \in [(2b'-1)k + 1,\ (2b'+1)k] \subseteq [0, Q).$$

Lower bound. We have

$$a \cdot q \geq ((2b'-1)k+1) \cdot q = (2b'-1) \cdot \frac{Q-1}{2} + q > b'Q - \frac{1}{2}Q,$$

where the last inequality holds since $2q + 1 - 2b' > 0$.

Upper bound. Moreover,

$$a \cdot q \leq (2b'+1)kq = \left(b' + \frac{1}{2}\right)(Q-1) < b'Q + \frac{1}{2}Q.$$

Combining the two bounds yields

$$b'Q - \frac{1}{2}Q < aq < b'Q + \frac{1}{2}Q,$$

and hence

$$\lfloor \frac{a \cdot q}{Q} \rceil = b'.$$

Therefore, $b = b'$.

Case 2: $b' = q$**.** This is the wrap-around case, for which

$$a \in [0, k) \cup [Q-k, Q).$$

We further distinguish two subcases.

1. If $a \in [0, k)$, then

   $$0 \leq aq < kq = \frac{Q-1}{2} < \frac{Q}{2},$$

   and thus $\lfloor \frac{aq}{Q} \rceil = 0$.

2. If $a \in [Q-k, Q)$, then

   $$(Q-k)q \leq aq < Qq.$$

   Since $(Q-k)q = Qq - \frac{Q-1}{2}$, it follows that

   $$q - \frac{1}{2} < \frac{aq}{Q} < q,$$

   and therefore $\lfloor \frac{aq}{Q} \rceil = q$.

In both subcases, we obtain

$$b \equiv \lfloor \frac{a \cdot q}{Q} \rceil \pmod{q} = 0.$$

□